Computers & Security 153 (2025) 104380

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Vulnerability defence using hybrid moving target defence in Internet of Things systems

Mohammed Tanvir Masud [a,*], Marwa Keshk [a], Nour Moustafa [a,*], Benjamin Turnbull [a], Willy Susilo [b]

[a] University of New South Wales, Canberra, Australia
[b] University of Wollongong, Australia

ARTICLE INFO

Keywords:
Moving Target Defense (MTD)
IoT security
Cyber threat actors
Networked assets
Security metrics
Attack surface dynamics
Risk assessment
Cyber resilience
Cybersecurity effectiveness
Security analysis

ABSTRACT

Cyber threat actors are increasingly targeting networked assets and critical infrastructure, with the potential for major socioeconomic impacts. Moving target defence (MTD) is a cyber defence paradigm that creates constantly shifting attack surfaces (i.e., vulnerabilities). It intends to make it more difficult for cyber adversaries to exploit systems, thereby increasing costs and chances of detection. There is a lack of research into the efficiency of combined MTD techniques, especially regarding several types of security considerations like time, cost, and effort. This gap is particularly significant in the Internet of Things (IoT) context, where security problems arise from its heterogeneous architecture. Moreover, MTD may result in the overutilization of network and system resources to enhance cybersecurity. We present a Vulnerability Defence method to address this issue using the three-layer Temporal Hierarchical Attack Representation Model (3-layer-THARM). This approach overcomes this difficulty by evaluating the safety of aggregated network states, considering security metrics in each state and the accessibility of network nodes and edges. Using this model, we can recognize probable attack scenarios in the context of Internet of Things (IoT) systems, conduct a thorough security analysis of the IoT system using well-defined security metrics, and assess the effectiveness of various defence tactics. This feature inherently introduces an additional level of security for the system. Furthermore, this model showcases the ability to identify potential attack pathways and effectively mitigate the consequences of such attacks. Our analysis reveals a noteworthy trend: combining MTD techniques from different categories, such as shuffle and diversity, generally produces more favorable outcomes, including a lower probability of attack success, lower attack risk and higher attack cost.

1. Introduction

Cybersecurity risks can emerge from various security weaknesses and vulnerabilities in operating systems (OSs), software, hardware or network applications. Cyber threats have grown swiftly and improved greatly, enabling them to avoid or neutralize standard security defensive mechanisms, including firewalls and intrusion detection systems (IDSs). IoT networks that are not well-designed open themselves to attacks like arbitrary command execution and firmware reprogramming. Such exploitable security weaknesses present systemic risks to networks and the resilience of the Internet, with examples such as the Mirai malware (Kolias et al., 2017) demonstrating the seriousness of the danger posed by hacked IoT devices. Mirai-infected devices launched a coordinated DDoS attack on Dyn, a prominent DNS service in the United States, jeopardizing its clients' income and reputation. Dyn saw an instant loss of about 8% of its customers (Neshenko et al., 2019).

Most recent vulnerability assessment research is focused on protecting the IoT environment (Sicari et al., 2015; Yu et al., 2013). Because these components evolved separately, previous methodologies frequently evaluate security in isolation, showing flaws in openness and standardization (Kim and Na, 2016). Current systems acquire data from the actual world via IoT systems, process this data using various IoT services, and then trigger actions in the physical world. As a result, a thorough assessment of safe data transfer should encompass not just IoT characteristics but also network security. Due to the restrictions of legacy network design, which is built on closed networks, extending to various services and interoperating with other devices or services is difficult. As a result, a separate approach to interconnecting complete networks is required (Kim and Na, 2016).

In the contemporary era of IoT, individuals are immersed in an environment characterized by a multitude of networked components.

* Corresponding authors.
E-mail addresses: mohammed_tanvir.masud@unsw.edu.au (M.T. Masud), nour.moustafa@unsw.edu.au (N. Moustafa).

https://doi.org/10.1016/j.cose.2025.104380
Received 25 August 2024; Received in revised form 10 January 2025; Accepted 10 February 2025
Available online 25 February 2025
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Despite the benefits of IoT and ubiquitous cloud computing, the domain faces the challenge of addressing numerous security issues. An example of a malevolent framework that has targeted the breadth of the nexus between cyber and physical is SmartThings (Liaskos et al., 2016). This attack framework has been utilized to carry out threats such as the unauthorized acquisition of door codes and attacks on standard PCs. Cloud computing security has become a significant concern for cloud providers. This challenge originates from the dependability of security within this expanding paradigm, especially when cloud organizations offer comprehensive services to their customers. As a result, several studies have investigated incorporating both reactive and proactive security measures to improve the security of cloud computing resources and services (Lv et al., 2023; Hu et al., 2018). Assessing how the cloud's security posture evolves when several security methods are combined is critical.

The static and passive nature of IoT devices creates a set of unique cybersecurity challenges, creating opportunities for attackers to find new vulnerabilities and launch attacks at their convenience. To mitigate these vulnerabilities, the cybersecurity paradigm of MTD has emerged as a proactive countermeasure that aims to change the attack surface dynamically. The MTD concept builds upon earlier work in fault tolerance, reconfigurable computing, and bio-inspired cybersecurity. Well-known systemic diversity techniques to boost reliability include n-version programming (NVP) (Avizienis, 1985), reconfigurable software (Compton and Hauck, 2002), and network reconfiguration (Baran and Wu, 1989); an early usage for cybersecurity protection is software/network diversity (Knight et al., 2016). The idea of MTD became more formalized in 2009 when the National Coordination Office for Networking and Information Technology Research and Development (NITRD) Program recognized MTD as a promising approach for achieving security by better utilizing the limited resources (Chong et al., 2009). More importantly, MTD introduces changes to the underlying infrastructure, making it impossible for an attacker to identify the location of an attack, which puts an immense amount of pressure on their resources. This strategy intends to increase the costs associated with attackers whilst decreasing those associated with the defense, all to create a more assertive defensive posture. This article highlights explicitly the MTD strategies that can be implemented to enhance security in IoT-based cloud computing environments.

The cybersecurity issues associated with IoT devices are often attributed to their static and passive characteristics, creating opportunities for attackers to examine possible vulnerabilities and carefully conduct attacks at their discretion. Moving Target Defence (MTD) is a proactive cybersecurity system that aims to correct weaknesses and strengthen security by making the attack surface dynamic. Implementing this approach offers an adequate amount of unpredictability to confuse attackers and complicate security-threatening endeavours (Sengupta et al., 2020). MTD tactics are anticipated to reduce defensive expenses and simultaneously raise the costs for attackers in terms of effort, time, and finances compared to alternative methods. To maintain economic viability and mitigate the risk of cyber-attacks, it is imperative that the recommended defensive MTD methods be both effective and efficient. This article utilizes MTD capabilities to bolster security in IoT-based cloud computing environments.

System security would be altered when an MTD technique is implemented for a particular purpose, such as vulnerability mitigation or enhanced resilience. Most proposed MTD strategies lack compelling data to support their stated effectiveness. Consequently, it is critical to evaluate the efficacy of MTD techniques using security metrics such as Return of Attack (RoA) and Attack Cost (AC), which assess security from the attackers' vantage point, as well as Risk (R) and Attack Success Probability (ASP), which may be deemed advantageous metrics from the standpoint of cloud providers. Security analysis is essential for assessing the comprehensive security aspects of a system. Assessing the efficacy of an MTD approach is crucial, as it may significantly influence the security status and overall ability to withstand the cyber assaults of a system. Security indicators such as Attack Cost and Return of Attack provide vital insights into the economic and strategic benefits offered by MTD (Moving Target Defence). Furthermore, risk metrics and attack success probability could be used to assess the prospective consequences on service dependability and client contentment. Hence, it is crucial to do a thorough security analysis before implementing MTD methods in any system.

Whether used alone or in combination, MTD approaches offer a viable route for improving security and system performance. However, as MTD techniques have grown, assessing their combined efficacy has become a difficult task (Alavizadeh et al., 2020). For example, redundancy improves service availability, but shuffle improves security dramatically. Nonetheless, it is critical to recognize that haphazard redundancy deployment can result in economic expenses and an enlarged attack surface. As a result, it is critical to carefully evaluate combinations of MTD approaches in order to achieve a delicate balance between security and cost concerns. The deployment of MTD methods in IoT-based cloud computing systems depends on the limits established by cloud providers. For example, adopting approaches such as OS diversity or virtual machine (VM) replication (as a redundancy strategy) entails the production of additional VM instances, thereby incurring expenses for cloud providers. MTD methods: shuffle and redundancy, can be installed in a cloud platform with minimal performance that impacts system interruption (Alavizadeh et al., 2018).

Research motivation. MTD techniques, used alone or together, can enhance security and minimize system disruptions. Combining methods like redundancy for better service availability and shuffle for increased security provides unique advantages. However, careful deployment is crucial to avoid economic costs and an expanded attack surface, especially with Redundancy. The strategic combination of MTD techniques aims to strengthen security and reliability, though synergy does not always guarantee improved system security due to potential operational conflicts. While prior research has suggested employing analytic modeling techniques to assess the performance implications of combining MTD methods, it does not address the challenge of potential conflicts arising from implementing diverse MTD approaches (Hong and Kim, 2015; Ge et al., 2017a; Ergenc et al., 2020). Therefore, a time-based trigger method is essential to assess resource availability for implementing the MTD strategy. MTD was developed as a proactive defence technique to counter cyberattacks by dynamically changing the attack surface of systems (Maughan, 2009). The fundamental design approach guiding the MTD method development has centered around three important issues (Cai et al., 2016): (1) what to move, (2) how to move, and (3) when to move. While existing research has focused on identifying what to move and how to move, the temporal component of when to move is considered as important (Alavizadeh et al., 2020; Ge et al., 2018). This aspect entails determining the best timing to transition from an MTD system's current state to a new state, hence nullifying any information or progress obtained by an attacker in the current state. MTD approaches need proactive adaptation, which is accomplished by periodically modifying settings or attributes at predetermined or stochastic intervals.

Furthermore, the complexity and variety of modern systems present challenges for monitoring, analyzing, and reacting to potential attacks in real-time. Modifying threat detection mechanisms to identify attacks directed at a dynamic target is difficult, as conventional detection methods, generally designed for static or relatively stable environments where the attack surface is well understood and does not change frequently (Carter et al., 2014; Aydeger et al., 2019). Ongoing research is devoted to developing sophisticated algorithms and tools that can effectively monitor the effectiveness of defence mechanisms in dynamic systems.

Although existing MTD techniques have effectively handled mitigating attacks in static environments, they still need to fulfill the requirements of IoT networks and the resource constraints of devices.

While previous efforts have only applied shuffling, diversity, or redundancy separately and combined to enhance security, no previous technique can adapt over time in response to a real-time attack on a particular region, as needed in an IoT system. On top of that, conventional MTD models lack timing mechanisms that choose the appropriate moment to deploy strategies in a manner that both minimizes the cost (in terms of defending resources) and maximizes the efficiency (in terms of generating security) of the defense methods used.

Previous works' limitations motivate us to design and construct a hybrid MTD model involving shuffle, diversity, and redundancy approaches to protect IoT networks. The three strategies deal with different security aspects of IoT: shuffling adds uncertainty by reconfiguring the network dynamically, and diversity increases robustness with diverse system components. At the same time, redundancy provides uninterrupted service continuity, even in the case of attacks on some nodes. These strategies work together to provide a synergistic multi-layered defense system, negating external and internal IoT threats (i.e., static vulnerabilities and constrained devices resources).

This model allows the use of MTD techniques in the defined intervals by adaptive application during the residency time-interval adjusting control in a very energy-efficient way for the resource-constrained setting of IoT environments. The synergistic innovation behind this approach is to deliver a balanced, anticipatory solution to mitigate against the level of IoT-induced harms (as assessed using metrics involving attack likelihoods and costs) while being resilient to attack both horizontally across threats otherwise needing to be addressed individually, and vertically across IoT survivability limitations. This hybrid approach delivers a balanced, resource-aware defense system that not only counters real-time IoT-specific threats but also ensures long-term system reliability and security.

Research objective. This work aims to design a hybrid Moving Target Defense (MTD) model that combines the advantages of shuffling, diversity, and redundancy strategies while considering the drawbacks of the current message MTDs for IoT networks. It includes a novel base interval control model time-based approach for such MTD techniques dynamically applied to tackle MTD land in a limited resource and, at the same time, in an optimal manner against IoT-specific vulnerabilities during real-time attacks.

This study aims to achieve the following specific objectives:

- To develop a new hybrid MTD method that integrates shuffling, diversity & redundancy methods to cater for the peculiar security requirements of IoT Networks, such as static configurations. The latter also encompasses assessing diverse MTD combination strategies and execution schemes over IoT-specific scenarios.
- To introduce and implement a time-interval control mechanism that dynamically regulates the deployment of MTD strategies, including VM placement for shuffling, OS diversity, and redundancy techniques, thereby optimizing resource use while maintaining security.
- The precise evaluation of the security risk of the system may be achieved by utilizing the 3-layer Temporal Hierarchical Attack Representation Model (3-Layer THARM). The present model demonstrates a proficient analysis of the attacker's behavior by employing the Host Bayesian Attack Graph, the Services on the Host Bayesian Attack Graph, and the Service Attack Tree.
- Experimental validation conducted in a simulated IoT environment, demonstrating significant reductions in ASP, increases in AC, and overall improvements in network resilience.
- A defense-adversary interaction model that evaluates how MTD strategies dynamically disrupt attacker workflows and force re-scanning, credential reuse, and other adversarial efforts, increasing their cost and reducing attack success rates.

This paper makes the following contributions to the multiple-time dictates of MTD in IoT networks, with the intention of establishing a relatively low-cost, adaptive defense system. Thus, it fosters the survival and robustness of the IoT system in the process of perpetrating real-time attacks in IoT.

The subsequent sections of this paper are organized as follows: In Section 2, we provide an overview of the background and relevant studies. Section 3 delineates the network configurations, encompassing discussions on the threat model, vulnerability defence framework, and proposed model. Section 4 elucidates the results obtained, while Section 5 addresses limitations and potential expansions. Finally, in Section 6, we draw conclusions.

2. Literature review

This section overviews the background and relevant research concerning MTD in IoT systems.

2.1. Moving Target Defense (MTD)

MTD is a method used to defeat adaptable enemies proactively, by increasing attacker cost and complexity. The main goal of this activity is to consistently navigate through different configurations within a network or computer system, modifying software, open network interfaces, and network settings in order to increase the level of uncertainty for potential attackers. This approach must be conducted while still maintaining consistent services for valid users. MTD approaches differ depending on how network configuration modifications are implemented (Zhuang et al., 2014a). Time-based MTD: This approach involves initiating MTD procedures at consistent time intervals. The MTD interval, which represents the time difference between successive MTD activities, can either be a fixed duration or adapt according to specific parameters. Event-based MTD: Event-based MTD triggers MTD operations in response to particular network circumstances or observable behaviors. Hybrid MTD: Hybrid MTD refers to integrating event-driven and time-based methodologies within certain MTD systems (Xu et al., 2023). Additionally, MTD categorizes its techniques into three distinct types, known as Shuffling, Diversity, and Redundancy, based on how they operate in response to the dynamics of target movement (Cho et al., 2020). Most evaluations of MTD techniques have considered outside attackers without taking into account any MTD approaches, such as event-based MTD or time-based MTD (Alavizadeh et al., 2020; Hong and Kim, 2015; Alavizadeh et al., 2022). The states and vulnerabilities that exist in modern IP networks are constantly shifting, making them extremely dynamic. Therefore, we considered time-based MTD to assess the effectiveness of these techniques.

2.2. Vulnerability assessment

The vulnerability assessment, integral to understanding the potential exploitation of vulnerable areas like software on hosts or communication interfaces between hosts, lays the groundwork for a comprehensive risk analysis (Zheng et al., 2022). This analysis is further enriched by vulnerability analysis, which merges vulnerability assessment and physical consequence assessment. Risk assessment employs two distinct approaches: the out-of-limits method, relying on predetermined criteria as the primary reference standard, and the event-driven method, which scrutinizes potential events. From the vulnerability assessment results, evaluating event probabilities encompasses empirical estimation and modeling calculations (Kolosok and Korkina, 2018). Predominantly, event consequences manifest as physical outcomes, encompassing economic and stability repercussions.

Conducting a vulnerability assessment mandates using a crucial mathematical model to abstract and quantify the attack process. Additionally, modeling analysis is pivotal, demanding a comprehensive

understanding of the network attack procedure (Sou et al., 2012; Chlela et al., 2017). To ensure a thorough evaluation of the effectiveness of Moving Target Defense (MTD) techniques, meticulous attention is required when selecting network security metrics. This assessment categorizes metrics into two primary classifications: static metrics, which provide a holistic evaluation of security resilience, and dynamic metrics, which assess the flexibility and effectiveness of MTD techniques against emerging threats. By judiciously choosing the most fitting assessment metrics, researchers can accurately gauge the efficacy of MTD approaches in enhancing network security. This cohesive approach integrates vulnerability assessment, risk analysis, and consideration of security metrics to provide a comprehensive understanding of network vulnerabilities and the effectiveness of defensive strategies.

The Attack Cost (AC) is a widely used static statistic that estimates how much it would cost an attacker to exploit a given vulnerability in a given system. To evaluate the efficacy of their MTD strategy from both the attacker's and defender's perspectives, Brown et al. (2023) used AC to assess MTD in a real-world setting.

2.3. Graphical Security Model (GrSM)

Ensuring the security of networked systems, including IoT, enterprise, and cloud environments (Ge et al., 2018; Nhlabatsi et al., 2018), is essential for mitigating the damage caused by cyberattacks. A common defense strategy involves using a Graphical Security Model (GrSM), such as an Attack Graph (AG) (Sheyner et al., 2002) or Attack Tree (AT) (Schneier, 2015), to evaluate a system's security and recommend improvements. GrSMs are valuable tools for assessing the effectiveness of Moving Target Defense (MTD) techniques (Hong and Kim, 2015). However, GrSM-based security assessments often face computational challenges, particularly in large-scale networks. To address this, the Hierarchical Attack Representation Model (HARM) was introduced as a formal, two-layer, graph-based model (Hong and Kim, 2012). Compared to other GrSMs, the three-layer Temporal Hierarchical Attack Representation Model (THARM) provides enhanced scalability and adaptability, making it especially suitable for complex IoT networks (Hong and Kim, 2013).

The integration of THARM with MTD strategies, including shuffling, diversity, and redundancy, is grounded in the layered defense paradigm. This approach aligns with established cybersecurity principles, as each MTD strategy targets specific IoT security requirements at different layers. Specifically, shuffling operates at the network layer to introduce unpredictability, diversity at the service layer to improve resilience, and redundancy at the application layer to ensure service continuity under attack. By combining these strategies within the THARM model, a structured framework is created that systematically applies each strategy at the appropriate layer, enhancing adaptability and resilience against evolving threats while efficiently managing IoT's constrained resources.

Several studies have explored the theoretical foundations, methodologies, and assessments of MTD (Sengupta et al., 2020). Hobson et al. (2014) and Zhuang et al. (2014b) highlighted that advancing MTD approaches depends on three primary factors: (1) element selection for movement, (2) space availability for movement, and (3) timing of movement execution. Additionally, Cai et al. (2016) identified critical considerations for applying MTD techniques: (1) identifying content to be relocated, (2) choosing the relocation method, and (3) establishing an execution schedule. This research primarily focuses on technical elements of MTD strategies, such as movement selection, strategy, and timing. However, the economic dimensions—specifically, the costs and effectiveness of MTD—remain relatively unexplored. Service providers must consider the total cost of security measures and evaluate MTD movements' cost-effectiveness in proportion to the security achieved.

The shuffle approach has been widely studied, with emphasis on its unique advantages. Jafarian et al. (2015) demonstrated IP shuffling's security and financial benefits. Danev et al. (2011) extended this model to secure cloud infrastructure, focusing on secure virtual machine migration. Penner and Guirguis (Penner and Guirguis, 2017) developed MTD solutions to enhance cloud security against Multi-Armed Bandit (MBA) attacks, evaluating effectiveness by measuring movement time between virtual machines (VMs). Nevertheless, much research has focused on performance optimization, often overlooking MTD's security impact. Limited research evaluates MTD's economic and security implications in IoT cloud environments.

Researchers have used the diversity approach to improve network and service resilience and increase attack complexity (Rehman et al., 2024; Anwar and Kamhoua, 2022; Chen et al., 2021). Applications include programming languages (Taguinod et al., 2015) and virtual servers (Huang et al., 2010), which have assessed the likelihood of successful attacks. Other diversity techniques, such as runtime program variations, aim to enhance resilience by implementing recovery mechanisms. Although prior studies have examined diversity in cloud security (Hong and Kim, 2015; Alavizadeh et al., 2020), economic considerations remain underexplored. Redundancy strategies, suggested to improve application-layer reliability and prevent web service attacks, have not yet been thoroughly analyzed for security (Cho et al., 2020).

To model and assess specific MTD strategies with THARM in IoT networks, several assumptions are made to manage constraints and scalability:

- Resource Constraints: IoT devices are assumed to have limited computational and memory resources, necessitating efficient MTD deployment to prevent resource depletion.
- Static Configurations and High Vulnerability: IoT networks often rely on static configurations, making them predictable and vulnerable. Shuffling at the network layer introduces necessary unpredictability to counteract this risk.
- Layer Isolation for Attack Paths: Each layer (network, service, application) is assumed to operate independently for security purposes, allowing MTD strategies to address vulnerabilities specific to each layer. This may limit inter-layer interactions but simplifies resource management and enhances energy efficiency.
- Event-Based Adaptability: THARM assumes that IoT threats can be mitigated with both time-based and event-driven triggers, balancing defense responsiveness with resource usage—a critical factor for resource-constrained IoT devices.

These assumptions provide a structured approach to deploying shuffling, diversity, and redundancy within the THARM framework, optimizing resource utilization while maintaining a robust, layered defense tailored to IoT environments.

Despite advances in MTD systems, a gap remains in modeling and assessing specific MTD strategies with consideration for services on VMs in IoT networks. While various MTD implementation and evaluation methods exist, few have addressed the integration of these services. This is an important research gap, as MTD effectiveness is closely related to the services within the network. For instance, prior research has presented analytical modeling tools to evaluate MTD performance (Connell et al., 2018, 2017; Alavizadeh et al., 2022). However, these models often make significant assumptions, such as an exponential distribution of services within empirically observed values, without examining specific events occurring during MTD execution.

3. Network configurations

The framework proposed in Fig. 2 contains n hosts, where $n = 8$. The eight hosts were selected to satisfy the system's design requirements to achieve optimal performance and resource allocation. This arrangement was intended to create a manageable yet complex environment to test the effectiveness of the MTD techniques, particularly the combination of Shuffle and Diversity strategies. In this situation, we develop trust in the system's hardware and the underlying logic of operating-system-level virtualization by utilizing the Trusted

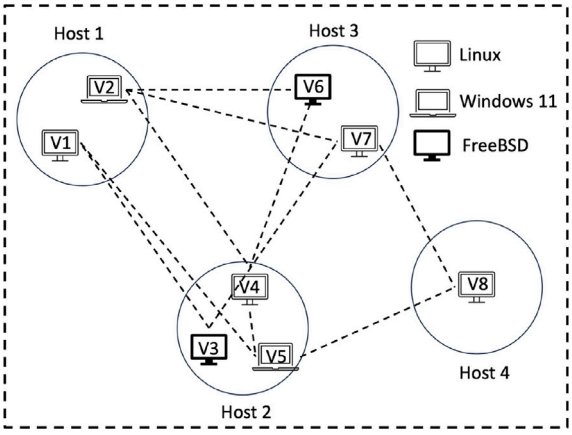


Fig. 1. Installing and configuring the virtualized host network.

Dynamic Logical Heterogeneity System framework (Ge et al., 2017b). The framework aims to discover all possible attack vectors in the IoT, evaluate the IoT's security status using security metrics, and evaluate the success of various defence tactics. Our focus remains entirely on assuming flaws in the operating system (while accepting the possibility of other vulnerabilities, such as application-level vulnerabilities, being included).

Because of security concerns, the components of the virtualized system are susceptible to frequent changes, including operations such as virtual machine (VM) creation, termination, and migration. Each of these virtual machines manages two services within the IoT network, where 'services' specifically refer to IoT-related functions, such as remote monitoring, data collection, or device management. Among them, V_8 exclusively connects to the database (DB), making it the prime target for potential attackers. This virtualized system is a model for modern networked configurations such as virtualized data centers and cloud computing environments. Within this example scenario, the following assumptions and setups are used (see Fig. 1), with the potential of their release:

- All hosts $V = V_1, V_2, \ldots . V_n$ remains continuously operational and active.
- Each host has a maximum capacity of accommodating a maximum of four VMs alone.
- $V_1, \ldots, V_4$, and V_6 have the capability to migrate across Host1, Host2, and Host3.
- V_5 has the capability to migrate across Host1, Host2, and Host4.
- V_7 has the capability to migrate across Host1, Host2, Host3 and Host4.
- Given its limited access solely to the database, V_8 becomes a prime target for attackers and remains ineligible for migration to other hosts.

3.1. Threat model

A threat model identifies potential vulnerabilities before they threaten the system, an essential step for developing IoT-specific security measures and strategies to mitigate risks (Zografopoulos et al., 2021). In this IoT network environment, we assume that a potential attacker is external to the private cloud and could exploit vulnerabilities present in IoT device operating systems to gain unauthorized access. The hypervisors are assumed to provide sufficient isolation for VMs, representing IoT devices, hosted on each server. The attacker's objective is to breach the database (DB) on host8, which stores critical IoT data.

We focus on vulnerabilities commonly associated with IoT environments, specifically bypass and remote access vulnerabilities in each of the three operating systems under consideration. Table 1 presents an overview of the identified vulnerabilities in Microsoft Windows 11, Linux, and FreeBSD.

These vulnerabilities are labeled w1–w6 for Windows 11, li1–li6 for Linux, and f1–f6 for FreeBSD. Each VM simulates an IoT device, running two active IoT services (e.g., data collection or remote monitoring), each with additional vulnerabilities, listed in Table 2. For each vulnerability, we considered the Common Vulnerability Scoring System (CVSS) base scores, impact values, and exploitability scores to assess the associated risks. Further details on these measurements are provided in Refs. (National Vulnerability Database (NVD), 2024; Common Vulnerabilities and Exposures (CVE), 2024). Our proposed model aims to counter the attacker's efforts to exploit these IoT-specific vulnerabilities through various MTD strategies, thereby enhancing the network's resilience.

3.2. The proposed framework for implementing MTD for vulnerability defence

Essential for an optimal MTD implementation, the suggested metho dology employs various strategic techniques, utilizing proactive, time-based execution methods. The shuffling, diversity and redundancy MTD strategies fits into the layered structure of the three layer Temporal Hierarchical Attack Representation Model (THARM) making them suitable for the specification of targeted applications to address the unique IoT security requirements at different levels. Specifically, redundancy and shuffling change the attack paths from network-layer, whereas diversity enhances the resilience at the service layer by providing different system configurations. This has allowed all strategies to work synergetically without working against each other in how they are operated.

Periodic triggers initiate MTD processes, exclusively designed to be proactive without reactive considerations. Upon receiving a periodic triggering signal, the system attempts to trigger an existing MTD instance or register a new one in line with the THARM model. All registered instances are added to the MTD queue, with the highest-priority instance activated and deployed unless there are suspension issues. For example, redundancy and shuffling both alter network paths and thus cannot run simultaneously. If redundancy is already active, shuffling will be suspended until redundancy completes. In contrast, diversity operates independently within the service layer, allowing it to function concurrently with network-layer strategies like redundancy or shuffling.

An MTD needs to gain access to the resource it serves after activation. Since a resource may be served by more than one MTD approach, the activated MTD starts by checking if the resource it is serving is available. The MTD will launch and occupy the resource until its execution is finished if the resource is accessible. If a resource is unavailable, the activated MTD will be halted until the resource is made available. During this process, domain names are responsible for managing the communication between hosts. Algorithm 1 demonstrates that when a DNS query is transmitted to resolve the domain name of an end host, the SDN controller intercepts the DNS response. The controller substitutes the real IP address of the target host with a virtual IP address.

The MTD strategies implemented in this experiment are used to exploit IoT-specific vulnerabilities, while realizing the efficient use of IoT resources in the context of deploying the THARM framework. In a traditional network, the SDN controller acts as a central decision-maker that creates rules for the traffic which in turn SDN-enabled switches store and follow using their flow table. Centralization enables dynamic and real-time changes in the network configuration, which are essential for a feasible implementation of MTDs. IP shuffling (or host address mutation), which is one of the main types of MTD methods having

Table 1
Operating system vulnerabilities in VMs.

OS vulnerability for IoT Services	CVE ID	IoT Vulnerability type	CVSS BS	Impact (I_v)	Exploitability (E_V)	Attack cost (C_v)
Windows 11 (w1 - w6)	CVE-2021-38631, CVE-2023-40023, CVE-2021-41338, CVE-2022-34709, CVE-2022-29103, CVE-2022-21894	RA, RA, RA, BP, BP, BP	2.1, 7.5, 2.1, 6.0, 4.6, 4.9	2.9, 3.6, 3.9, 5.2, 6.4, 6.9	0.39, 0.39, 0.29, 0.08, 0.39, 0.39	1.07, 1.20, 0.27, 1.47, 0.13, 0.40
Linux (li1 - li6)	CVE-2023-25136, CVE-2022-2663, CVE-2023-38432, CVE-2021-4135, CVE-2022-2503, CVE-2021-32958	RA, BP, RA, BP, BP, RA	6.5, 5.3, 9.1, 5.5, 6.9, 2.1	4.2, 1.4, 3.9, 1.8, 0.6, 2.9	0.22, 0.39, 0.52, 0.36, 0.58, 0.39	1.27, 1.55, 1.45, 0.64, 0.36, 1.00
FreeBSD (f1 - f6)	CVE-2014-8612, CVE-2010-4755, CVE-2023-38432, CVE-2010-4754, CVE-2019-9499, CVE-2023-3326	RA, RA, BP, RA, BP, BP	4.6, 4.0, 9.1, 4.0, 6.8, 9.8	2.9, 1.4, 3.9, 2.9, 6.4, 5.9	0.39, 0.80, 0.52, 0.80, 0.86, 0.39	1.17, 0.93, 0.99, 1.25, 0.81, 0.99

Notes: Remote Access = RA, Bypass = BP, CVE data sourced from the national vulnerability database (National Vulnerability Database (NVD), 2024) and CVE (Common Vulnerabilities and Exposures (CVE), 2024).

Table 2
Vulnerabilities in the services of VMs.

IoT services vulnerability	CVE ID	IoT vulnerability type	CVSS BS	Impact (I_s)	Exploitability (E_s)	Attack cost (C_s)
$SN_1 - SN_{14}$	CVE-2023-27389, CVE-2020-5536, CVE-2021-1540, CVE-2021-33215, CVE-2017-9382, CVE-2020-3624, CVE-2019-2341, CVE-2021-40831, CVE-2022-42055, CVE-2022-25686, CVE-2020-26079, CVE-2021-35097, CVE-2020-15858, CVE-2021-30331	RA, BP, BP, DT, DT, OV, OV, BP, DT, OV, RA, BP, DT, OV	7.2, 5.8, 6.0, 4.0, 4.0, 4.6, 4.6, 6.0, 6.5, 9.8, 4.0, 6.8, 3.6, 2.1	5.9, 6.4, 6.4, 2.9, 2.9, 6.4, 6.4, 6.4, 3.6, 5.9, 2.9, 5.9, 4.9, 2.9	0.12, 0.65, 0.68, 0.80, 0.80, 0.39, 0.39, 0.68, 0.28, 0.39, 0.80, 0.09, 0.39, 0.39	1, 0.9, 0.2, 0.1, 0.8, 0.3, 1.1, 1.2, 1.5, 1.4, 0.5, 0.2, 0.9, 1.2

Notes: Remote Access = RA, Bypass = BP, Directory traversal : DT, Overflow = OV, CVE data sourced from the national vulnerability database (National Vulnerability Database (NVD), 2024) and CVE (Common Vulnerabilities and Exposures (CVE), 2024).

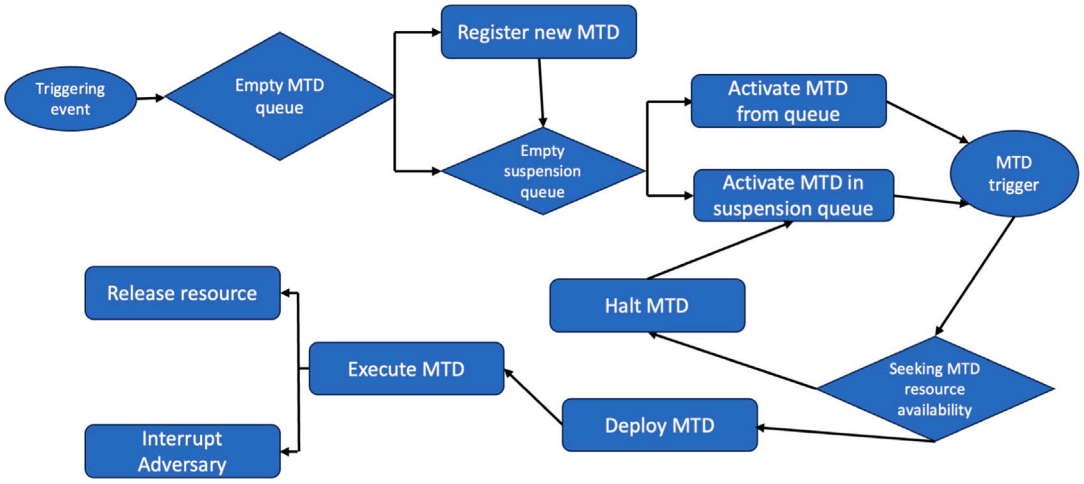


Fig. 2. An operational framework of MTD actions for vulnerability defence.

a lot of research work done is based on that principle. This process dynamically maps hosts on the network to assigned or temporary vIPs (virtual IPs) that are temporary in nature and their assignment fails previously success IP discovery, defeating network reconnaissance.

The SDN controller assigns an rIP and a corresponding vIP to the host while maintaining the mapping between them (Algorithm 1). Once the MTD is fired, new vIPs will be assigned to the hosts, and the controller will update the updates for the SDN-assisted switches. It abstracts the IP mutation from hosts and end-users and ensures packets are forwarded to the correct vIPs seamlessly and automatically. The translation of rIPs to vIPs is done at the network edge, ensuring that the packets are translated at the SDN switches right before they hit the destination hosts. Most legitimate clients will be able to resolve the new vIPs through an authenticated DNS service, and attackers will be none the wiser about the hosts they previously discovered being invalid. One interesting aspect of the MTD is that it can be activated periodically (e.g., every ten minutes), as a response to a network event, or both shorter intervals offer stronger protection, but the performance costs also increase.

For example, the experiment incorporates redundancy, diversity, and shuffling as MTD strategies. Redundancy and shuffling affect the network layer's attack paths and cannot be activated simultaneously. However, diversity operates at a lower layer and can be deployed concurrently with the other strategies without interference. Through this experiment, the SDN controller's role in dynamically managing MTD strategies, particularly IP shuffling, is demonstrated, enhancing network security by complicating the attacker's ability to exploit network configurations.

3.3. Experimental environment, parameters and validation

This section introduces the experiment, since it involves multiple setups, parameters, configurations and a number of security metrics to evaluate. They are crucial for creating a controlled environment and assessing the influence of the suggested MTD approaches in an IoT environment. Due to the static and resource-constrained nature

Table 3
Symbols, explanations, and formulas: a reference for the reader.

Symbols	Explanation	Formula
V_i	The set of virtual machines in the IoT cloud network	$V_i = \{V_1, V_2 \dots V_n\}$
s_i	The set of services in the IoT cloud network	$s_i = \{s_1, s_2 \dots s_n\}$
V_i^o	The ith VM has the oth vulnerability associated with the OS	$V_1^o = V_1^1, V_1^2 \dots V_1^n$
V_i^s	The ith VM has the sth vulnerability associated with the service	$V_1^s = V_1^s, V_1^{s+1}$
G_{t_i}	At time t_i edge sets between nodes	$G_{t_i} = \{G_{t_1}, G_{t_2} \dots G_{t_n}\}$
ap	An attack path in the IoT network is defined as a sequence of consecutive VMs in the cloud that extends from the entry node within the edge sets to the target node.	$ap = (V_1, V_2 \dots, V_n)$
AP	All possible cloud attack paths leading to the target node, starting from an entrance point within the edge sets.	$AP = ap_1, ap_2, \dots, ap_n$
ASP_{V_i}	Attack success probability associated with ith VM	Eq. (1)
R_{vi}	Attack risk associated with ith VM	Eq. (2)
R_{ap}	The cumulative risk value of all VMs along an attack path (ap).	Eq. (2)
R	The cumulative risk value for all possible attack paths (AP) across the edge sets.	Eq. (2)
AC_{vi}	Attack cost associated with ith VM	Eq. (3)
AC_{ap}	The cumulative attack cost of all VMs along an attack path (ap)	Eq. (3)
AC	The cumulative attack cost for all possible attack paths (AP) across the edge sets.	Eq. (3)
RoC_{vi}	Return of attack ith VM	Eq. (4)
RoC_{ap}	The cumulative return of attack of all VMs along an attack path (ap)	Eq. (4)
RoC	The cumulative return of attack for all possible attack paths (AP) across the edge sets.	Eq. (4)
C_{v_i}, D_{v_i}, B_{v_i}	Closeness of certainty, degree of certainty, betweenness of certainty associated with the ith VM across the input edge sets G	Eqs. (5), (6), (7)

Table 4
System parameters and initial VM security metrics.

Parameter	Value
Number of hosts	8
Maximum VMs per host	4
Maximum service per VM	2
VM vulnerability types	Remote Access, Bypass
Migration capability	Enabled for V1–V7, fixed for V8
IP addressing scheme	Dynamic with shuffling intervals
Defense trigger interval	10 m
VM Metrics (V1, V2, V3, V4, V5, V6, V7, V8)	ASP: (0.975, 0.986, 0.99, 0.98, 0.9745, 0.99, 0.97, 0.95) AC: (8.17, 6.54, 8.64, 9.97, 7.34, 8.84, 8.57, 6.27) AR: (47.19, 36.68, 51.07, 47.64, 40.7, 52.12, 47.57, 33.73)

of IoT devices, these parameters are critical in creating a controlled yet realistic environment that reflects the unique challenges of IoT networks. Here we summarize the parameters and their default values for this experiment, as seen in Table 4. This includes the number of hosts, virtual machine configuration, important security parameters and initial value of security metrics for each virtual machine.

The initial security metrics – Attack Success Probability(ASP), Attack Cost(AC) and Risk(AR) – of individual VM are calculated based on its vulnerabilities, exposure level and criticality in the network. A thorough methodology of how these metrics were calculated is provided in Algorithm 2.

The MTD defenses proposed in the framework were validated through a controlled experiment designed to mimic realistic conditions on an IoT network. Eight virtual hosts (V1 through V8) each managed two services, with host V8 fixed to simulate a critical database. Periodic triggers were programmed to initiate shuffling, diversity, and redundancy strategies for moving target defense (MTD) in an adaptive, efficient manner. Key metrics like attack success probability, attack cost, and attack risk were automatically calculated at intervals using the threat and harm assessment risk model (THARM). Attackers were modeled after techniques in the cyber kill chain and MITRE ATT&CK, mirroring real threats (Hutchins et al., 2011). Triggers deliberately shuffled intervals to balance protection with resources. Tests included widespread and focused attacks aiming to breach many or specific high-value hosts. Vulnerabilities were prioritized by return on attack, emphasizing those with greatest impact. Successful exploitation let adversaries control internals through command channels to connected devices. Watching strategies interact with adversaries showed how movement disrupted plans. IP shuffling and path alteration terminated live links and concealed pathways. Service and operating system diversification modified configurations, forcing fresh scans and adaptation to new flaws. These fluid defenses complicated enemy progress, slowing or thwarting compromises. Data after each change evaluated reduced probability of success, greater costs, and strengthened security overall, demonstrating the approach's skill in addressing IoT weaknesses.

3.4. Proposed method- three-layer THARM

We used the Graphical Security Model (GsrM), specifically the Hierarchical Attack Representation (HARM) (Enoch et al., 2022), to represent the IoT network for conducting a security study and evaluating the effects of Moving Target Defense (MTD) approaches. The Three-layer Temporal Hierarchical Attack Representation Model (THARM) structures the system into three analytical layers – the upper layer (U), the service layer (S), and the lower layer (L) – to provide a comprehensive framework for evaluating MTD strategies. The top layer of THARM illustrates the connections and interactions among various IoT devices and services, while the lower layers focus on constructing and analyzing attack trees to evaluate potential threats and vulnerabilities within the IoT architecture.

- U (Upper Layer): Represents the network-level attack graph of Virtual Machines (VMs), offering a topological overview for assessing MTD strategies.
- S (Service Layer): Captures the attack paths and vulnerabilities specific to IoT services, evaluating MTD effectiveness on service-level resilience.
- L (Lower Layer): Uses attack trees to detail vulnerabilities within individual IoT services and devices, enabling granular analysis of MTD impact on component security.

In this setting, the THARM framework is applied to evaluate the effectiveness of different combinations of MTD strategies. Following this, we analyze changes in security metrics resulting from implementing MTD techniques at a subsequent stage. To enhance understanding of the calculation process, we break it down into four specific definitions, providing illustrative solutions.

The framework design corresponds to standard IoT architectures, including sensor-to-cloud interconnection, device management, and data stream administration across constrained IoT nodes. This tiered method enables us to evaluate the impact of MTD strategies on network security within a realistic IoT environment. By addressing the unique security limitations of IoT devices, the model helps identify probable attack paths and supports targeted mitigation using MTD strategies such as IP address shuffling, OS diversity, and service redundancy, which are essential for strengthening IoT network defenses.

Definition 1. This 3-Layer THARM is represented by a 5-tuple TH = (N, U, S, L, C), where N is a set of network states, U signifies the upper-layer attack graph of VMs, S denotes the upper-layer attack graph of services, L represents the lower-layer set of attack tree (AT), and C stands for $C \subseteq \{(s_i \leftrightarrow V_i), (V_i \leftrightarrow AT)\} \forall s_i \in S, V_i \in U, AT \in L$.

The THARM for this instance can be represented as $TH_{hS} = \left(N_{hS}, V_{hS}, S_{hS}, L_{hs}^{N_{hs}}, C_{hs}\right)$ where $N_{hS} = n_{hs,t}$ signifying the network states at the given time instance t. $V_{hS} = (attacker, V_1, V_2, V_3, \ldots . V_8)$ constituting a limited array of attack VMs within the network at that time. Similarly, $S_{hS} = (attacker, S_1, S_2, S_3, \ldots . S_{14})$ is a finite collection of attack services present within the network at that time. $L_{hs}^{N_{hs}}$ comprises a set of attack trees meticulously constructed to singularly encapsulate the vulnerability specifics of each node N_{hS} in the upper layer during the given time instance t, while C_{hs} denotes the correlation connecting the upper layer VMs with their corresponding lower layer Attack Tree models. This signifies that for every VM within the upper layer, an associated Attack Tree exists in the lower layer.

Definition 2. In the upper layer THARM of VM, an attack graph is represented as a 2-tuple $U_{t_i} = (V_{t_i}, E_{vt_i})$ at time t_i, where V_{t_i} represents a finite set of VM, and $E_{vt_i} \subseteq V_{t_i} \times V_{t_i}$ denotes a set of edges.

The upper layer of the THARM at time t_1 for network state n_{hs,t_1}, can be represented as $U_{hs,t_1} = (V_{hs,t_1}, E_{hs,t_1})$. Here, $V_{hs,t_1} = V$, E_{hs,t_1} is a set of edges defined as $(attacker, V_1), (attacker, V_2), (V_1, V_3), (V_1, V_5), (V_5, V_8), (V_3, V_7), (V_2, V_6), (V_2, V_4), (V_4, V_5), (V_5, V_8), (V_2, V_7), (V_7, V_8)$.

Definition 3. In the upper layer THARM for services, an attack graph is represented as a 2-tuple $S_{t_i} = (s_{t_i}, E_{st_i})$ at time t_i, where s_{t_i} represents a finite set of services of network nodes N_{t_i}, and $E_{st_i} \subseteq N_{t_i} \times N_{t_i}$ denotes a set of interconnected edges.

The upper layer of the THARM for services at time t_1, corresponding to network state n_{hs,t_1}, can be denoted as $S_{hs,t_1} = (s_{hs,t_i}, E_{ss,t_1})$. where, $s_{hs,t_i} = S$, and $E_{ss,t_1} = (attacker, s_1), (attacker, s_2), (attacker, s_3), (attacker, s_4), (s_1, s_2), (s_1, s_2)(s_5, s_6), (s_1, s_2)(s_9, s_{10}), (s_3, s_4)(s_7, s_8), (s_3, s_4)(s_{11}, s_{12}), (s_3, s_4)(s_{13}), (s_{14}), (s_{12}, s_{13})(s_7, s_8), (s_7, s_8)(s_9, s_{10}), (s_9, s_{10})DB, (s_{13}, s_{14})DB$.

Definition 4. The lower layer of the THARM encompasses a collection of AT_s. This structure characterizes the vulnerabilities set VN_{t_i} and SN_{t_i} at time t_i, associated with each VM and service. In this context, an AT can be represented as a 4-tuple $AT_j = \left(A_{t_i}, B_{t_i}, c_{t_i}, root_{t_i}\right)$, where AT_j is an element of AT_s. Here, $A_{t_i} \subseteq VN_{t_i} \cup SN_{t_i}$ forms a set of vulnerabilities. B_{t_i} comprises a collection of gates. $c_{t_i} \subseteq \left\{b_{t_i}^j \rightarrow e_l\right\} \forall b_{t_i}^j \in B_{t_i}, e_l \in A_{t_i} \cup B_{t_i}$ represents a correspondence between gates and vulnerabilities as well as other gates. The $root_{t_i} \in A_{t_i} \cup B_{t_i}$ signifies the root node of AT_j.

The THARM of lower layer for network state n_{hs,t_1}, the set of Attack Trees is represented as $L_{hs,t_1} = \left\{l_{V_1}, l_{V_2}, \ldots, l_{V_8}\right\}$. The Attack Tree corresponding to a V_1 is denoted as $l_{V_1} = \left(A_{V_1,t_1}, B_{V_1,t_1}, c_{V_1,t_1}, \text{root } V_{V_1,t_1}\right)$ where $A_{V_1,t_1} = (V_1^{\circ}) \cup (V_1^s, V_1^{s+1})$, $B_{V_1,t_t} = \{OR_1\}$, $c_{V_1,t_t} = \left\{\left(OR_1, (V_1^1, V_1^2, \ldots, V_1^n) \cup (V_1^s, V_1^{s+1})\right)\right\}$, and $\text{root}_{V_1,t_1} = OR_1$.

3.5. IoT network security metrics calculation

Network security metrics (NSMs) facilitate a quantitative evaluation of the overall resiliency of interconnected systems against malicious attacks (Ramos et al., 2017). Security metrics are computed for the services and VMs linked to the IoT network. Certain higher-level metrics are obtained from those at lower ones during the security analysis phase. This analysis assesses the probability of attack success, attack cost, attack risk, return of attack, and the importance measures. The probability of success is between 0 and 1, whereas cost and risk are between 0 and 10. The higher the value within this range, the more likely an attacker will be able to exploit a vulnerability in the system. The symbols, explanations, and mathematical formulas for calculating security metrics such as Attack Success Probability (ASP_{V_i}), Attack Risk (R_{v_i}), Attack Cost (AC_{v_i}), and centrality measures are summarized in Table 3. These components provide a foundational reference for evaluating the security posture of the IoT network under varying MTD strategies.

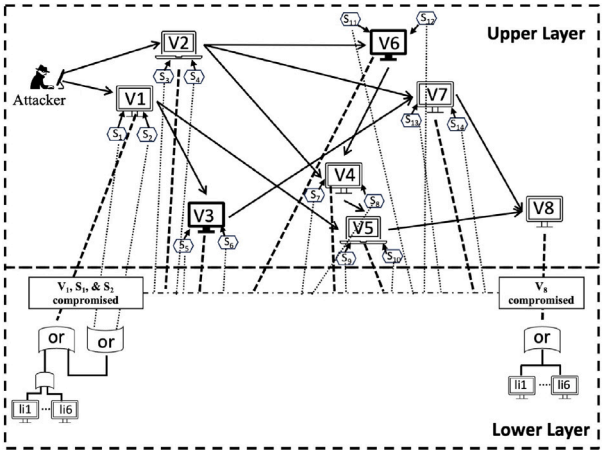


Fig. 3. Configuration of the upper and lower layers of THARM with distinct vulnerabilities.

Algorithm 1 IP shuffling process with MTD Integration

Triggering Event:
INITIALIZE: Check if the MTD queue is empty
if MTD queue is empty **then**
  Register new MTD
  Activate MTD from a queue
  Perform IP Shuffling Process
else
  Activate MTD in the suspension queue
  Perform IP Shuffling Process
end if
IP Shuffling Process:
for each packet p from host h_i to host h_j **do**
  if p is a match to DNS response for host h_j **then**
    Set $dstAddr(p) \leftarrow vIP(h_j) : dstPort$
  end if
  if p is at the source SDN-Switch **then**
    Set $scrAddr(p) \leftarrow vIP(h_i) : srcPort$
    Set $dstAddr(p) \leftarrow rIP(h_i) : dstPort$
  else if p is at the destination SDN-Switch **then**
    Set $dstAddr(p) \leftarrow rIP(h_j) : dstPort$
    Set $scrAddr(p) \leftarrow vIP(h_j) : srcPort$
  end if
end for
for each service s of host h_i (total 2 services) **do**
  Select a new vIP for each service s of h_i
  Update flow table entry
end for
Interrupt adversary if necessary
Release resource

Network security metrics adhere to Algorithm 2 for evaluation, where all possible edge sets are denoted by G. The edge set can be modified for different MTD strategies, such as shuffling or introducing redundancy at different times (t_n). Algorithm 2 outlines the calculations for network security metrics, accompanied by equation numbers from Eq. 1 to Eq. 7.

Algorithm 2 Security Metrics Calculation

Input: All possible Edge sets G, considering time t_i edge-sets G_{ti}
Output: ASP_{v_i}, AR, AC, RoC, NC, C_{v_i}, D_{v_i}, B_{v_i}
/* Calculate attack success probability for each VM with input $E_v, E_s, I_v,$ & I_s as in Eq. (1) */
$ASP_{V_i} = 1 - \left(\prod_{v_{j,\omega_i} \in V_{i,\omega_i}} \left(1 - E_{v_{j,\omega_i}}\right) + \prod_{s_{j,k_i} \in V_{i,k_i}} \left(1 - E_{s_{j,k_i}}\right)\right)$
/* Generate paths with the inputs G, S_s & T_r */
AP = all_simple_paths (G, S_s, T_r)
for ap in AP **do**
  for V_i in ap
    /* Calculate the attack risk associated with ith VM as in Eq. (2) */
    $R_{vi} = \prod_{v_{j,\omega_i} \in V_{i,\omega_i}} \left((E_{v_{j,\omega_i}} \times I_{v_{j,\omega_i}}\right) + \prod_{s_{j,k_i} \in V_{i,k_i}} (E_{s_{j,k_i}} \times I_{v_{j,\omega_i}}))$
  R = append(append(R_{vi}, R_{ap}), R)
    /* Calculate the attack cost associated with ith VM as in Eq. (3) */
    $AC_{vi} = (\prod_{v_{j,\omega_i} \in V_{i,\omega_i}} \left(C_{v_{j\cdot wi}}\right) + \prod_{s_{j,k_i} \in V_{i,k_i}} \left(C_{s_{j\cdot ki}}\right))$
  AC = append(append(AC_{vi}, AC_{ap}), AC)
    /* Calculate the return of attack associated with ith VM as in Eq. (4) */
    $RoC_{vi} = R_{vi}/AC_{vi}$
  RoC = append(append(RoC_{vi}, RoC_{ap}), RoC)
  /* Calculate closeness of certainty associated with the ith VM using the input G, as in Eq. (5) */
  $C_{v_i} = (|g| - 1) \times \left[\sum_{j=1}^{|g|} d\,(G)\right]^{-1}$.
  /* Calculate degree of certainty associated with the ith VM using the input G, as in Eq. (6) */
  $D_{v_i} = degree(V_i)/(|g| - 1)$.
  /* Calculate betweenness of certainty associated with the ith VM using the input G, as in Eq. (7) */
  $B_{v_i} = \sum(\sigma(G, \mathrm{V_i})/\sigma(G))$
  end for
end for

3.5.1. Attack success probability

This metric relates to the probability that attacks will be successful. It measures, for example, the likelihood of a target being found or successfully accessed by an attacker or compromising a system component (or defence). This metric combines the degree of system vulnerability with the attacker's access and attack execution capabilities, accounting for the attacker's available resources. It assesses the probability that an attacker can access a system component effectively.

The attack success probability (ASP) for each VM (V_i) and its associated service (s_i) can be donated as ASP_{vi}. The metric signifies the probability of an attacker successfully compromising the target through all potential network states at a specific time t_i. The ASP_{vi} can be determined by considering the exploitability values associated with the vulnerabilities present on the VM and services. The calculation of ASP_{v_i} is outlined in Algorithm 2 Eq. 1, considering the exploitability value of the jth VM as V_j and their associated OS and service exploitabilities, denoted as E_{w_i} and E_{k_i}, respectively, with their corresponding values shown in Tables 1 and 2.

3.5.2. Attack risk

The creation of the THARM opens the door to acquiring essential security metrics. Attack Risk (AR) is a holistic assessment of the IoT network's overall risk within this context. This construction entails collecting data concerning the network's VMs and associated services. Furthermore, it encompasses a meticulous evaluation of vulnerabilities for each VM and its services, drawing from a dedicated vulnerability database like the National Vulnerability Database (NVD), as detailed in Tables 1 and 2. To determine AR, we leverage both layers of the THARM framework. The upper layer represents VM connectivities, while the lower layer captures vulnerabilities. The computation of AR is elucidated in Algorithm 2 Eq. 2. Here $E_{vj\cdot w_i}$ and $I_{v_j\cdot w_i}$ represent the exploitability and impact values of the jth vulnerability present on the OS(wi) within VM V_i. Similarly, $E_{sj\cdot w_i}$ and $I_{s_j\cdot w_i}$ denote the exploitability and impact values of the jth vulnerability associated with the services.

3.5.3. Attack cost

The cost associated with an attacker exploiting vulnerabilities on a VM and its associated services is termed the Attack Cost (AC). We extend this metric to derive the comprehensive AC for the IoT network. We employ the upper layer of THARM for AC computation. Tables 1 and 2, consider the cost linked to exploiting vulnerabilities in VMs. The calculation process for AC is detailed in Algorithm 2, Eq. 3 Here, $C_{vj\cdot w_i}$ and $C_{sj\cdot k}$ represent the attack costs for the OS and services of the VM (V_i).

3.5.4. Return of attack

Return of Attack (RoA) is a security measure from the attacker's perspective. RoA is the ratio between how much an attack costs and the benefit it provides. A higher RoA number means that attackers are more likely to take advantage of weaknesses, which means they are more likely to launch attacks. Algorithm 2 Eq. 4 shows how to calculate a VM's RoA value from the ratio of its risk value to the cost of an attack.

3.5.5. Network certainty

Centrality metrics leverage the topological data within the graph to evaluate the importance of individual nodes. Network centrality measures (NCMs) are employed for pinpointing crucial VMs in the higher layer of the HARM. Some of these metrics, like degree centrality, rely solely on the local information of nodes. The normalization of closeness centrality is achieved by dividing it by the factor $(n - 1)/(|g| - 1)$, where 'n' denotes the number of nodes in the connected component of the graph that includes the node. When the graph is not completely connected, this approach calculates closeness centrality for each connected component individually. It adjusts it according to the size of that particular component, as shown in Algorithm 2 Eq. 5. The following algorithm is used to compute closeness centrality. It determines the shortest distance between nodes n_i and n_j, where 'g' is the total number of nodes in a linked graph. Similarly, the degree of certainty and betweenness can be calculated as shown in Algorithm 2 Eqs. 6 & 7.

3.6. Security-driven MTD model

A cyber resilience system supported by an intrusion defence system (IDS) is made more secure with MTD. MTD serves as an intrusion prevention tool by reducing the likelihood of possible intrusions into the system. The attack surface is effectively modified by various MTD approaches, including Internet Protocol (IP) mutation, changes in network topology, platform migration, and software stack shuffles. As a result, this reduces the chances for attackers to discover weak points in the system and their attempts to compromise it (Huang et al., 2022). MTD can aid with the detection of possible attackers by making their actions more complex. Because of this complexity, monitoring can collect intelligence on attack patterns and behaviors, ultimately assisting intrusion detection systems in enhancing attack detection.

Developing effective security strategies requires the formation of a protective model. The protective model can evaluate the current system and determine the optimal deployment of MTD techniques based on the system's requirements, restrictions, and limits. The Security-Driven MTD Model follows Algorithm 2 for evaluation, utilizing security metrics information. Algorithm 3 delineates the calculations for various security-driven MTD methods, accompanied by equation numbers ranging from Eqs. 8 to 11.

Algorithm 3 Security-Driven MTD Model

Input: Edge sets G, AR, AC, RoC, NC, C_{v_i}, D_{v_i}, B_{v_i}
Output: $H^m = (U^m, S, L, C)$, $H^d = (U, S, L^d *, C)$, $H^r = (U^r, S^r, L^r, C)$
/* at time t_i Select specific VM from $C_{v_i}, D_{v_i}, B_{v_i}$, as in Eq. (8) */
$V_i^* = rank(\arg\max(C_{v_i}, D_{v_i}, B_{v_i}))$
*/ Implement shuffling strategy (H^m) on the selected VM using V_i^* Eq. (9) */
for each edge $e_{i,j} \in G_{ti}$:
  If $i = V_i^*$ or $j = V_i^*$, remove $e_{i,j}$ from U.
*/ Calculate the shortest path from the edge for creating a new edge */
  Create a new edge $e_{x,j}$ and add it to U
end for
/ Implement diversity strategy (H^d) on the selected VM using V_i^* Eq. (10)/
foreach $V_i \in \{V_1 \dots V_i\}$ **do**
  if $V_i = V_i^*$
  $V^D \leftarrow$ set backup OS to the selected V_i
end for
/ Implement redundancy strategy (H^r)on the selected VM using V_i^* Eq. (11)/
foreach $V_i \in \{V_1 \dots V_i\}$ **do**
  if $V_i = V_i^*$
  $V^r \leftarrow$ set new vulnerabilities to the selected V_i
  create $L^r\left(V^r, G, V_{n+r}\right)$
  add V_{n+r} into VM
  foreach $e_{i,j} \in G$ do
  If $i = V_i^*$ or $j = V_i^*$, added new edge $e_{i,j}$ in U.
  Create a new edge $e_{x,j}$ and add it to U
  end for
end for

3.6.1. IP variation (IPV)

This section explores the dynamics of IP address changes within a network over time, a critical aspect of Moving Target Defense (MTD) strategies. At any given time t_i, the set of active IP addresses, denoted by $\mathbb{IP}_{t_i}$, is defined as:

$$\mathbb{IP}_{t_i} = \left\{ h_i \cdot IP_{t_i} \mid \begin{matrix} h_i \cdot IP_{t_i} \text{ is a currently used IP of} \\ h_i \in \mathbb{H} \end{matrix} \right\}$$

where h_i is a host in the network $\mathbb{H}$.

An IP variation strategy periodically changes host addresses at T intervals, obfuscating the network topology. This temporal variation is observed through network conditions $\mathbb{IP}_{t_i}$ at different time points t_i. To measure the variability between network states, we define the generalized IP Shuffling measure:

$$IPV_{t_i,k} = \begin{cases} \frac{|\mathbb{IP}_{t_i} - \mathbb{IP}_{t_{i-1}}|}{|\mathbb{IP}_{t_i}|} & \text{for } k = 1 \\ \frac{\sum_{i=2}^{m} \frac{|\mathbb{IP}_{t_i} - (\bigcup_{j=1}^{k-1} \mathbb{IP}_{t_j})|}{|\mathbb{IP}_{t_i}|}}{m-1} & \text{for } k > 1 \end{cases}$$

where "–" denotes the difference operation, "|" signifies set cardinality, and k represents the number of observable states. For $k = 1$, this equation reduces to the basic IP Shuffling IPV_{t_i} between consecutive time points. For $k > 1$, it calculates the average IP Shuffling over multiple observable states.

Using this generalized expression, we can rigorously assess the effectiveness of IP-shuffling MTD techniques in dynamically altering network configurations, thereby enhancing the resilience of the network against potential attacks.

3.6.2. MTD shuffling strategy

Shuffle-based MTD methods involve the reconfiguration of existing network setups. In this instance, we have implemented the VM live migration (VM-LM) process as a shuffling technique. Cloud providers can activate VM-LM, enabling a VM to migrate from one host to another if the target host has sufficient free storage capacity. Importantly, this approach maintains vulnerability information, with its impact primarily limited to the upper layer of the HARM. Migrating from one host to another can influence the overall security strategy. By evaluating network security metrics, we examine the consequences of implementing the shuffle technique in the IoT cloud network.

Four hosts are shown in Fig. 4 as options for VM live migration (VM-LM), two of which are shown in Figs. 4(a) and (b). Implementing migration techniques involves two key aspects: (1) selecting the 'what to move', i.e., the VM to be deployed, and (2) determining the 'where to move', the destination for the VM (Cho et al., 2020). Since the attacking adversary can directly access host one, our objective is to select a VM on host one with a lower attack success probability, which can be determined using Algorithm 2. We introduced a replacement method, depicted in Algorithm 3, which identifies the shortest path in the upper layer of THARM and relocates the selected VM to that destination.

The main goal of this replacement technique is to extend the shortest attack path possible for attackers from the entrance point (the Internet) to the final target (the database), in conjunction with the selection of VMs housed on host one with lower attack success rates. Fig. 3 shows that the easiest attack path for attackers looking to compromise the database is V2 → V7 → V8, with V2 as the starting point and ending at V8. The migration of V2 and the subsequent creation of two new reachability connections are then depicted in Fig. 4(a). Crucially, it should be mentioned that only V1 and different intermediate nodes may provide an attacker with access to V2. The migration of VM-LM from host 2 to host 4 is also illustrated in Fig. 4(b) for another short route, V1 → V5 → V8, where VM-LM was deployed.

The shuffle function is applied to THARM as follows: Within the THARM framework, let $M(H, V_i^*, t_i)$ be a migration function, where V_i^* represents the specific VM selected for migration at time t_i. $M(H, V_i^*, t) = H^m(t_i)$ represents the result of the migration function. We define $H^m(t_i)$ as (U^m, S, L, C), where U^m is the modified AG at time t_i resulting from the migration of V_i^* in the upper layer of HARM, and $U^m = (N, E')$ is the representation of U^m, with E' being a subset of $N \times N$. It is important to emphasize that the migration function $M(H, V_i^*, t_i)$ preserves S, L, and C since it exclusively influences the reachability of VMs in the top layer of THARM at time t_i.

3.6.3. Path Shuffle Variation (PSV)

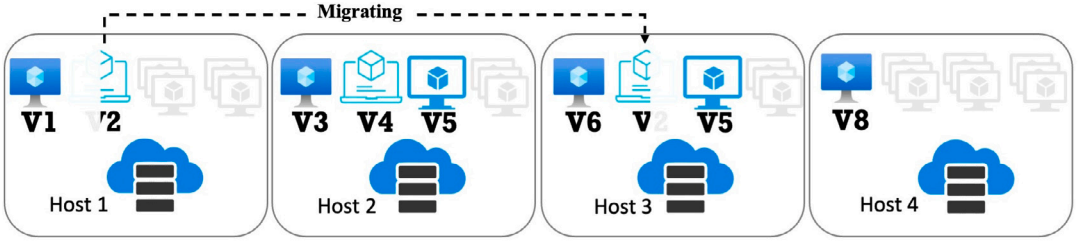
This indicator evaluates the modification in attack paths to assess the impact of implementing MTD. As shown in Fig. 4, MTD shuffling frequently alters the network attack path. We model these network dynamics over time at different points t_i with AG_{t_i} and calculate every possible path of attack at each t_i. For example, a graph of network reachability can be used to determine attack paths at different times (i.e., $t_1, t_2, \dots, t_n$). A set of every potential attack path at t_i is represented by $A\mathbb{P}_{t_i} = \{ap_{t_i} \mid ap_{t_i} \in AG_{t_i}\}$, where ap_{t_i} represents the path taken by an attacker to reach a target at a specific time t_i.

Similar to computing IPV, we calculate the immediate PSV at time t_i, denoted by PSV_{t_i}, based on $A\mathbb{P}_{t_i}$. The overall PSV for the duration of the time window $[t_1, t_n]$, with PSV being a real number between $[0, 1]$, is given by:

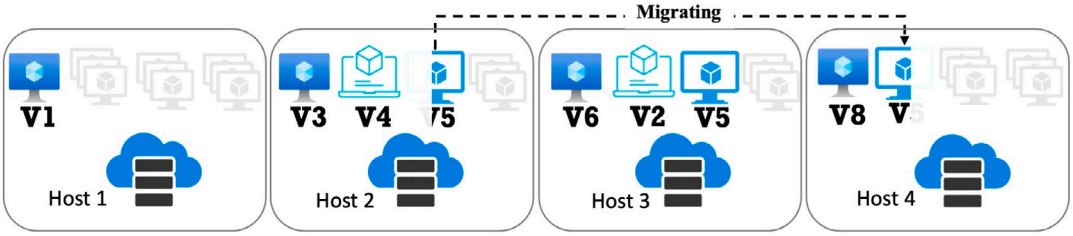
$$PSV = \frac{1}{n-1} \sum_{i=2}^{n} \frac{|A\mathbb{P}_{t_i} - A\mathbb{P}_{t_{i-1}}|}{|A\mathbb{P}_{t_i}|}$$

3.6.4. MTD diversity strategy

This strategy involves implementing alternative methods to construct system components while keeping the same capability. For example, alternative routing pathways or switching between platforms with different software component implementations might involve shifting between software stacks and hardware configurations (Cho et al.,



(a) Migration of V2 at time t_1.



(b) Migration of V5 at time t_2

Fig. 4. VM Migration: Transitioning from one host to another.

2020). We implement the diversity strategy by diversifying the operating system instances (OS Diversity) for VMs within the IoT network. The reachability of the networked system and the interdependencies among its components are assumed to be unaffected by the Diversity strategy. This strategy primarily aims to maintain the same level of system operation as previous versions, even if it updates vulnerability information extensively. This affects the foundation of the HARM. If the Diversity strategy is applied to VMs at the OS layer, resulting in OS diversity, then the Attack Tree (AT) for VM's bottom layer must be updated. This is due to the unique security risks posed by each operating system.

An instance of OS diversity applied to V1 involves its initial operation with the Linux OS at the upper layer, followed by a transition to Windows 11. It renders the attacker's gathered information for exploiting the current system's vulnerabilities ineffective and simultaneously introduces novel vulnerabilities, compelling attackers to grapple with these new security challenges. Consequently, attackers must invest additional time and resources to acquire information and exploit these new vulnerabilities. We assume that this cloud network can employ a backup OS with a lower attack success probability, making it more challenging for attackers to exploit the VM.

Formalization of the Diversity strategy is applied to THARM as follows: In the context of the THARM framework, consider $D(H, V_i^*, t_i)$ as a Diversity function, with V_i^* representing the specific VM chosen for Diversity at time t_i. The equation $D(H, V_i^*, t) = H^d(t_i)$ denotes the outcome of the diversity function. We define $H^d(t_i)$ as (U, S, L^d*, C), where L^d* is the modified AT at time t_i resulting from the diversity of V_i^* in the lower layer of HARM. Here L^d* is the backup OS each VM can donate as $L^*_{hs,t_1} = \left\{l_{V_1}, l_{V_2}, \ldots, l_{V_8}\right\}$. It is important to emphasize that the migration function $D(H, V_i^*, t_i)$ preserves S and C since it exclusively influences the reachability of VMs in the lower layer of THARM at time t_i.

3.6.5. MTD redundancy strategy

Redundancy approaches entail generating several replicas of system or network components, all providing identical functionality at either the network or application layer (Tan et al., 2023). The primary goal of this method is to improve the reliability and availability of the system by implementing redundant means to provide the same services in case certain network nodes or system components are compromised. This method enhances the resilience of a system or network in the presence of insider threats. We implemented the redundancy strategy by including and duplicating the chosen VMs offering the same services within the IoT network, thus introducing an additional node.

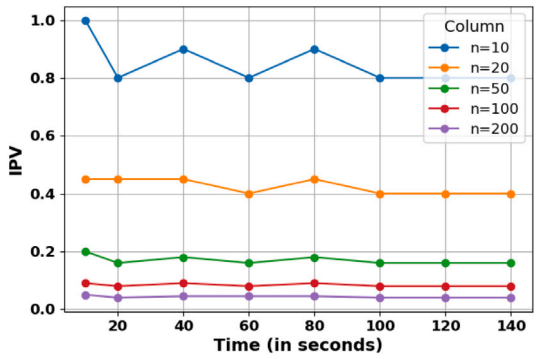
Formalization of Redundancy technique, wherein the redundancy function is employed on THARM in the following manner: Define the redundancy function that converts H into H^r: $R(H, V_i^*, r)$. In this instance, V_i^* denotes the chosen VM for virtual machine replication, which is carried out r times, contingent upon $1 \leq k \leq n$.

After V_i^* is replicated in the upper layer of THARM, U^r is an altered AG. As a result, H^r may be represented as a 4-tuple $H^r = (U^r, S^r, L^r, C)$. $U^r = \left(V', E'\right)$ is one way to represent it. In this case, E' indicates the new connectivity between VMs in the higher layer, and VM' represents a group of VMs, including the just replicated VMs.

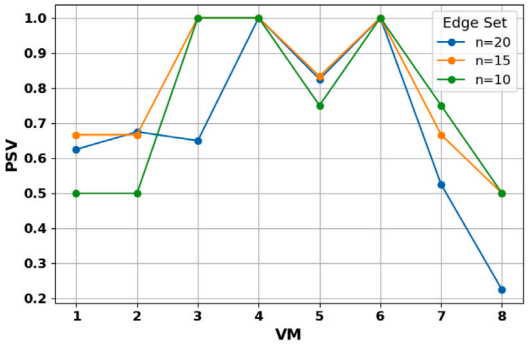
More vulnerabilities in the lower layer of THARM could be introduced by developing and deploying additional HARM VMs in the higher layer. Given that V^r represents a collection of vulnerabilities arising from the replication of a virtual machine in the upper layer of HARM, $L^r = \left(V', G, \text{root}\right)$, where $V' = V \cup V^r$, may be used to update the lower layer of HARM.

3.6.6. Integrating shuffle, diversity, and redundancy

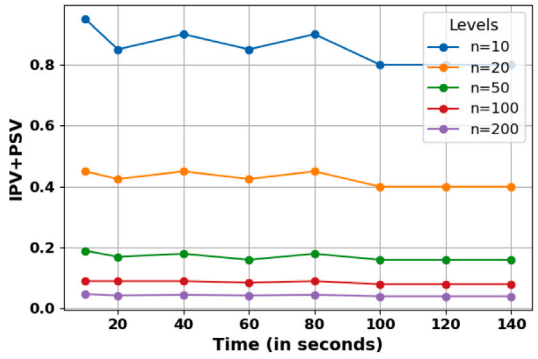
The integration of MTD strategies – Shuffle, Diversity, and Redundancy – hardens the system against multiple layers of vulnerabilities, leveraging the advantages of one through the other and making it harder for attackers to take advantage of weaknesses. Shuffle randomizes system configurations by changing attack paths or component movements (VMs), enabling an unpredictable network that hinders the attacker's mapping capabilities and targeting specific system resources. Applied at a lower layer, diversity introduces heterogeneity, utilizing different variants of components (for example, different operating systems), which requires the attacker to face a broader range of attacks to exploit the same known vulnerability on multiple platforms, hence making exploiting known attacks more complex. Redundancy provides fault tolerance by maintaining multiple instances of critical components or services so that the system can still be available in case one of the instances fails or goes down. By utilizing Shuffle, Diversity, and Redundancy together, they form a layered defense; Shuffle makes it challenging for an attacker to track the paths, diversity adds a layer of complexity using variants, and Redundancy guarantees system availability. It makes the system unpredictable and resilient, increasing the Cost (time, resources, and effort) of attacks and thereby reducing the probability of success; this multi-layered approach makes a critical difference.



(a)



(b)



(c)

Fig. 5. Analysis of IPV and PSV with varying host numbers (n), showing IPV with respect to time (a), PSV with respect to VM (b), and combined IPV+PSV with respect to time (c).

We demonstrated that using the shuffle strategy reduces system risk and AC value. It would, however, increase the possibility of an attacker creating a scheme to breach the system. However, diversification boosts AC without affecting system risk. We used the shuffling approach with VM-LM and the diversity technique with OS diversity to investigate the combined benefits of these strategies. For each VM in the network, we installed both VM-LM and OS diversity at the same time. Our data show that combining shuffle and diversity boosts AC while decreasing RoA value. Although integrating redundancy with OS variety and shuffling introduces additional network vulnerabilities, it improves network dependability, as evaluated by NCMs. As a result, combining the three basic MTD techniques – shuffle, diversity, and redundancy (S+D+R) – not only reduces AR and RoA but also enhances overall network reliability.

4. Results and analysis of MTD approaches

In this section, we delve into the outcomes and examine the MTD approaches applied within the circumstances of IoT systems. Our discoveries are founded on shortcomings inherent in the IoT context, and the MTD strategies were assessed under conditions that mirror the explicit difficulties of IoT systems. These conditions incorporate asset limitations, static device arrangements, and normal IoT risks like unapproved distant access and misuse of device vulnerabilities.

By surveying the execution and viability of MTD procedures inside IoT systems, we expect to show how these methodologies can ease IoT-particular dangers and upgrade the general security of IoT networks. The assessment zeroes in on how MTD systems can disturb potential assault pathways, increment the aggressor's expense and multifaceted nature, and diminish the shot of effective attacks against IoT services. The outcomes demonstrate that the utilization of progression and assorted qualities can viably undermine progression aims and increment the work included with recognizing frail spots along these lines, forestalling assaults. In any case, asset limitations are deterrents to applying MTD on some confined IoT devices.

4.1. Evaluation of IP shuffling and path shuffling variability

The simulation results utilizing the IPV measure are shown in Fig. 5(a), which also shows the implications of changing the number of hosts over time. In IoT networks, where devices often have fixed IP addresses and are susceptible to scanning and reconnaissance attacks, IP shuffling introduces necessary unpredictability to protect against such threats. The IPV metric exhibits minimal variability following each shuffling event. Moreover, the incidence of Internet Protocol Version (IPV) is greater in networks with smaller sizes than in networks with larger sizes, mostly because smaller networks have a reduced probability of having overlapping network states. During the 0 to 140 s window, the Ingress Protection Value (IPV) for a sample size of 20 is 0.4. This means that the IP-shuffling method renders 40% of reconnaissance attempts ineffective throughout this period. Fig. 5(b) illustrates the impact of attack path diversity on specific virtual machines (VMs) across various network sizes (n). In the context of IoT, path shuffling helps to obscure communication paths between IoT devices and services, thereby reducing the risk of attackers successfully mapping the network and exploiting vulnerable devices. The PSV value for VM 3 increases from 0.5 to 1 when $n = 10$. Additionally, the PSV value rises significantly as the number of hosts decreases, reaching 1 (indicating 100% variability) for a host size of 10. Similarly, Fig. 5(b) shows the impact of variability on specific VMs across various network sizes (n) on PSV. The analysis results using the IPV+PSV measure are depicted in Fig. 5(c), showing the combined impact of both IPV and PSV over time across different network sizes (n).

This metric is designed to capture both the effectiveness of IP shuffling in preventing reconnaissance (IPV) and the variability in attack paths (PSV). The graph indicates that smaller networks exhibit higher IPV+PSV values, reflecting greater resilience due to less overlap in network states and more diverse attack paths. For instance, in a network with $n = 10$, the IPV+PSV value remains relatively high throughout the analysis period, demonstrating consistent resistance to reconnaissance and attack path predictability. Conversely, larger networks with $n = 200$ show lower IPV+PSV values, indicating less variability and higher chances of repeated network states. This suggests that smaller networks benefit more from IP-shuffling techniques in terms of overall security as captured by the combined metric.

4.2. Evaluating the efficiency of shuffling

We use VM-LM as the underlying technique for implementing Shuffle in our simulation. In the IoT context, shuffling helps to obscure device configurations and communication patterns, making it more difficult for attackers to target specific IoT devices and services. Each VM migration from one host to another may influence the system's overall security. We examine how the Shuffle approach affects network security metrics. The shuffling strategy disrupts potential attack paths targeting IoT services, thereby enhancing the security of IoT devices against common threats such as remote exploitation and unauthorized access. The output of the Shuffle function on the produced THARM for the IoT network, in which the most important VM is chosen to be shuffled (V_i^*). We selected particular VMs for migration based on Algorithm 3 outlined in Eq. 8. According to the results of Eq. 8, VM V_5 obtained the highest rank compared to other VMs. Hence, at time t_1 the shuffling strategy is implemented at the upper layer, AG, resulting in an impact on $U^m = (N, E')$. After this process, the new edge nodes and VMs' connectivity can be described as $E' = \{(A, V_1), (V_1, V_2), (V_1, V_3), (V_2, V_3), (V_2, V_4), (V_2, V_5), (V_1, V_5), (V_1, V_6), (V_6, V_4), (V_4, V_5), (V_5, V_8), (V_7, V_8), \ldots.\}$. Following the specification of the Shuffle technique, we implement Shuffle on a VM and then analyze the security metrics to evaluate the deployed technique's impact in the cloud environment.

Deploying $M(H, V_i^*, t_i)$ has the potential to impact the edges in the higher layer of HARM. Consider the new attack path that results from the use of Shuffle. Let us examine V_5^* at time t_1, employing the shuffling strategy where V_5 migrates from host 2 to host 4, leading to the creation of a new attack path, denoted as $ap_1 = \{(A, V_2), (V_2, V_7), (V_7, V_5), (V_5, V_8)\}$ where the risk associated with the path is $R_{ap1} = R_{V2}+R_{V7}+R_{V5}+R_{V8} = 16.28+8.67+12.58+5.62 =$ **43.15**, Similarly, $AC_{ap1} = AC_{V2}+AC_{V7}+AC_{V5}+AC_{V8} = 37.5+43.2+46.4+39.4 =$ **166.5** and $RoC_{ap1} = R_{ap1}/AC_{ap1} =$ **0.259**.

We represent the collective network security metrics following the implementation of Shuffle techniques on the IoT network as R_s, AC_s, and RoC_s. After deploying Shuffle on V_5 to relocate it from host 2 to host 4, we observe the following values for the network security metrics: $R_s = 167.38$, $AC_s = 666.2$, and $RoC_s = 1.01$. Consequently, the enhancements in all security metrics for VM migration between hosts are depicted in Fig. 6. These results indicate improvements in all security metrics for V_5 shuffle from 2 to 4, including R_s, AC_s, and RoC_s, as a direct consequence of implementing Shuffle techniques within the IoT environment. From this, it can be determined that the attacker has fewer incentives to target that particular IoT cloud configuration when RoC_s is low. On the other hand, a higher RoC_s means that the attacker is more likely to attempt the same attack again, given it was previously successful.

4.3. Evaluating the efficiency of diversity

This work contributes to the Diversity technique as $H^d(t_i)$, where a specific VM is chosen according to network security metrics, as illustrated in Algorithm 3, Eq. 8. The Diversity technique is then applied to the VMs under the diversity Algorithm 3 Eq. 10. We then evaluate the security metrics once every Diversity scenario is deployed. We swap out the current OS for every VM for a different one. More specifically, we use Linux as the backup OS of Windows 11, Windows 11 backup OS FreeBSD, and FreeBSD backup Linux, as each OS has different vulnerability values, as shown in Table 1. Unlike the Shuffle approach, adopting Diversity does not modify or alter any attack pathways in the THARM's top layer. Instead, adding new vulnerabilities adds heterogeneity to the THARM's lower layer. As a result, the security metrics associated with each attack path may change.

For instance, let us contemplate an attack path present in the IoT network at time t_1, denoted as 1st attack path $(ap_1) = \{(A, V_2), (V_2, V_4), (V_4, V_5), (V_5, V_8)\}$ and we selected particular VM V_5 for diversity based on Algorithm 3 outlined in Eq. 8. After deploying the diversity for V_5 the attack risk for the path $R_{ap1} = R_{V2}+R_{V4}+R_{V5}+R_{V8} = 16.28 + 12.47 + 8.9 + 5.6 =$ **43.31**, similarly, $AC_{ap1} =$ **134.134** and $RoC_{ap1} =$ **0.32**. We represent the collective network security metrics following implementing diversity techniques on the IoT network as R_d, AC_d, and RoC_d. After deploying diversity on V_5, we observe the following values for the network security metrics: $R_d = 163.73$, $AC_d = 624.73$, and $RoC_d = 1.04$. Consequently, we have created Fig. 6 to illustrate the diversity strategy in security metrics. Utilizing an alternate OS for diversity implementation can substantially influence security metrics, especially regarding the reliance on vulnerability data that the new OS introduces.

In addition to the diversity strategy illustrated in Fig. 6, we also explore its impact on ASP across eight different paths comparing the results before and after the application of diversity as a defense mechanism. Fig. 7 presents these results where the blue bars represent the ASP before diversity, and the orange bars represent the ASP after diversity.

In general, the diversity technique appears to reduce the ASP for most paths, indicating improved security and a lower likelihood of successful attacks. For instance, Paths 3, 5, and 6 show notable reductions in ASP, suggesting a significant improvement in defense. However, paths like Path 7 and Path 8 exhibit little to no change, implying that diversity had minimal or no impact on those specific paths. Overall, this chart highlights the potential effectiveness of diversity in enhancing system security, though the technique's success may vary across different attack paths.

4.4. Evaluating the efficiency of redundancy

We present the result of implementing the Diversity technique as $H^r(t_i)$, where a particular VM is selected for deploying the redundancy strategy. The redundancy technique is subsequently applied to these VMs under the guidance of redundancy Algorithm 3 in Eq. 11. After each deployment of the redundancy scenario, we assess and analyze the security metrics. We represent the Redundancy technique as creating r replicas on a VM. It is important to note that the Redundancy technique impacts HARM's upper and lower layers. This impact is due to introduction of new connections between VMs and creating a new set of vulnerabilities for each newly generated VM.

Consequently, both layers of THARM need to be updated to accommodate these changes. The impacts of the redundancy method on individual VMs are shown in Table 5. As an example, following the implementation of the redundancy strategy on V_5, a new attack path has emerged, denoted as $attack_path = \{(A, V_2), (V_2, V_4), (V_4, V_5'), (V_5', V_8)\}$. Upon the deployment of redundancy for V5, the attack risk for the path remains consistent with the diversity approach, with VM $Rap_1 =$ 43.31. Likewise, AC_{ap_1} is recorded as 134.134, and $RoC_{ap_1} = 0.32$. We express the aggregate network security metrics resulting from applying redundancy techniques to the IoT network as R_r, AC_r, and RoC_r. Following the deployment of redundancy on V_5, the network security metrics exhibit the following values: $R_r = 260.84$, $AC_r = 878.3$, and $RoC_r = 1.46$.

These strategies are particularly relevant in IoT environments, where devices often run on outdated software and may not receive regular updates, making them vulnerable to exploits. By introducing diversity and redundancy, we can enhance the resilience of IoT networks against targeted attacks. Our analysis reveals that combining MTD techniques from different categories, such as shuffle and diversity, generally produces more favorable outcomes as seen in Fig. 6, including a lower probability of attack success, lower attack risk, and higher attack cost.

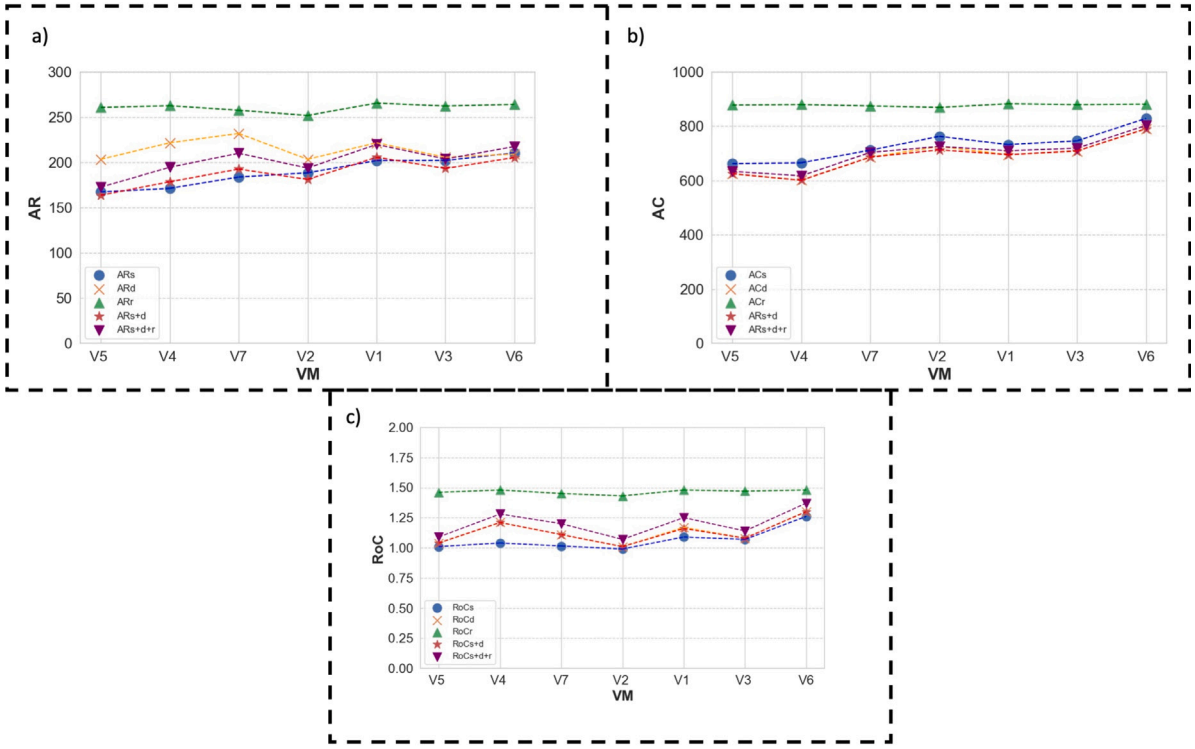


Fig. 6. The changes in network security metrics following the implementation of shuffling strategy, diversity strategy and redundancy on each VM in the IoT cloud network.

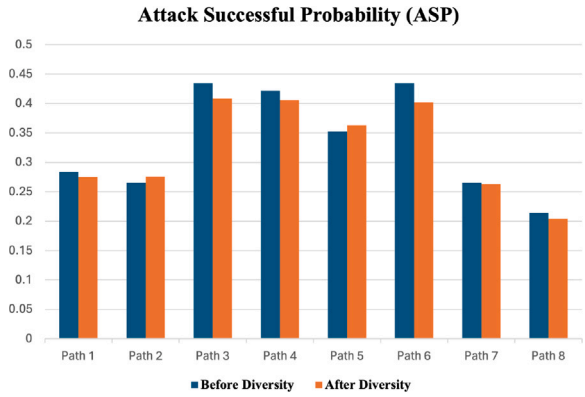


Fig. 7. Impact of Diversity on Attack Success Probability (ASP). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4.5. Evaluation in large cloud

We examine the efficacy of single and combination MTD methods in a larger operational environment to comprehend their scalability and performance. This thorough assessment offers insights into the scalability of our suggested MTD approaches and tactics. It also illuminates the changing nature of cloud security posture, demonstrating how security concerns shift with variations in cloud scale. The network comprises 400 VMs with a cloud-bound node. Within this setup, we posit that only a select few VMs serve as entry points to the system, specifically functioning as front-end servers directly connected to the Internet. Attackers can exploit weaknesses in each VM, map out attack routes, and ultimately get access to the target node. All VMs in the cloud-band nodes mostly utilize Windows as the main OS and Linux as the backup OS. The OS's vulnerabilities are detailed in Table 1.

We apply multiple MTD approaches to cloud bands of varying sizes of VMs and assess their efficacy. We evaluate the efficiency of each MTD combination by analyzing pertinent security metrics throughout this deployment. The R and RoA rates show higher decreases with technique D, as depicted in Fig. 8(a) and (b), implying that technique R produces better results for increasing attack costs within the established constraints of the cloud and system. With increasing cloud-band sizes, we see a linear growth in all indicators. Although the measure R performs poorly for D and ROA alone, its effectiveness improves when combined with S+D+R. The combo of S+D regularly shows considerable overall performance.

In this section, we examine the influence of time on MTD strategies, exploring how dynamic adjustment in methods such as shuffling, diversity, and redundancy can be countered over time for increased resilience in defence. This time-sensitive approach offers a more complete view, unlike previous studies that ignore time as an influencing variable.

4.6. Comparison with the present study

Table 5 presents a comparison of existing MTD literature, including potential combination MTD strategies. This comparison evaluates MTD approaches, combination tactics, and security metrics. We investigate the integration of MTD approaches, acknowledging that each technique may be efficient in particular situations. Many recent papers have focused solely on evaluating the effectiveness of MTD strategies without addressing any specific vulnerability defence models. Another limitation of recent articles is their exclusive focus on VMs, neglecting the impact of associated services, which also play a significant role in the threat model.

We analyze Attack cost (AC), Attack Risk (AR), Return on Cost (RoC) and Attack Success Probability(ASP) in metrics across time internals, as shown in Fig. 9, to monitor how each of the metrics responds against MTD adaptations. However, as the need for resources to continue attacking defences grows, Attack Cost usually increases exponentially

Table 5
Comparison of MTD papers based on integration of MTD strategies and security metrics.

Ref.	MTD techniques	Combination strategies	Security metrics					Vulnerability defense model	Considered node	
			AP	AR	AC	RoC	*NC*		VM	Service
This paper*	S, D, R	✓	✓	✓	✓	✓	✓	✓	✓	✓
Alavizadeh et al. (2020)	S, D	✓	✓	*x*	✓	✓	*x*	*x*	✓	*x*
Alavizadeh et al. (2022)	S, R	x	✓	*x*	✓	*x*	*x*	*x*	✓	*x*
Alavizadeh et al. (2021)	S, D	x	✓	*x*	*x*	✓	*x*	*x*	✓	*x*

Notes: Shuffling = S, Diversity = D, Redundancy = R.

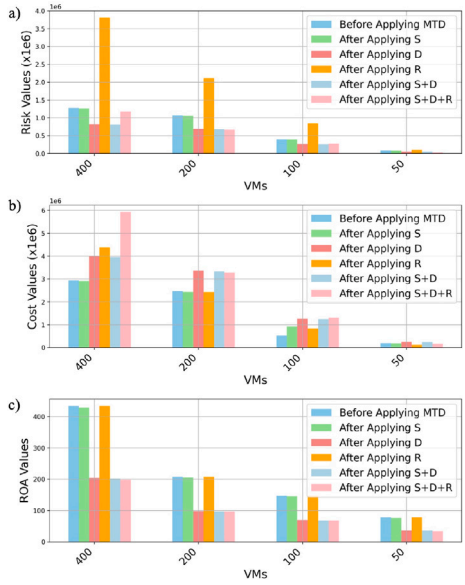


Fig. 8. Comparing network security metrics before and after implementing four MTD strategies.

over time. On the contrary, meanwhile, Attack Risk, RoC, and ASP, as illustrated in Fig. 9, are dynamic properties that reflect a constantly changing security environment provided by a time-based MTD approach. It is specifically connected with the steps in the cyber kill chain each of the subsequent MTD changes means that attackers will have to change their strategies: in turn, it needs more money, impacts success rate and affects efficiency.

Alternatively, the MTD studies in Table 5 do not consider time. Such works tend to be largely static, analyzing shuffling, diversity or redundancy in isolation from one another and ignoring temporal dynamics. The absence of a time dimension ignores the fact that reconfigurations change security stances over time. This figure shows that when MTD strategies are time-sensitive, security outcomes can be distinguishably different from those calculated for static approaches. This difference then proves the contribution of dynamic strategies toward improved MTD effectiveness.

5. Discussion and future research opportunities

The implementation of MTD techniques within the THARM framework exemplifies a novel tactic for mitigating security vulnerabilities, particularly concerning IoT environments. Our experiment illustrates how combining MTD methods including heterogeneity, redundancy, and network path manipulation enhances networked systems' resilience against attacks, providing a dynamic defense that thoughtfully adapts and acts proactively.

One of the key discoveries from our experiment involves the potency of diversification. By leveraging multiple operating systems (Linux, FreeBSD, Windows 11) for virtual machines, we introduced dissimilarity within the IoT network. Unlike redundancy and shuffling which alter top layers, diversity introduces irregularity at lower strata, adding intricacy and expanding the time and assets required for an attacker to successfully compromise the system.

Moreover, redundancy and shuffling play vital roles in transforming network arrangements. While redundancy confirms the network can perform even if parts of the infrastructure are compromised, shuffling modifies network routes, thereby making it difficult for aggressors to pursue and exploit the system. These techniques, when used jointly, reinforce defenses by preventing attackers from gaining a stable foothold, compelling them to continuously adapt to evolving system configurations.

The integration of SDN additionally amplifies the effectiveness of these MTD strategies. SDN facilitates centralized management and permits real-time changes to network settings. Through the SDN controller, we exhibited how dynamic alterations to flow rules, for example employing vIPs, can seamlessly assist IP shuffling without impacting end-user interaction. This flexibility combined with automatic deployment of new flow rules in SDN-enabled switches ensures protections are responsive to real-time dangers, without significant disturbance to network services.

Our experiment also underscores the practical application of these MTD techniques when confronting reconnaissance attacks, where attackers gather information regarding the network to plan their intrusion. By using IP shuffling, we display how a constantly evolving network arrangement makes it difficult for attackers to maintain an accurate view of the network. The random assignment of vIPs not only invalidates formerly discovered IP addresses but also reduces the potency of attacks reliant on network scanning, thereby furnishing an extra layer of defense against such reconnaissance activities.

The findings demonstrate how different MTD approaches work depending on the particular situation. Comparing the combined approaches' results with individual MTD results demonstrates that using many categories of MTD techniques is more effective than using fewer categories. Even though the study offers insightful information, it is important to acknowledge several limitations that might have affected some results. This section describes these limitations and suggests possible directions for future research to ensure the robustness of future studies.

For the purpose of experimental evaluation, MTD techniques were simulated to function on every node in the IoT network in this study. However, implementing MTD techniques across all nodes could potentially cause resource strain and service disruptions. To overcome this obstacle, it is crucial to identify and selectively reconfigure critical network nodes or subnets while deploying MTD techniques to minimize service interruptions. This focused strategy would be implemented via network analysis, traffic profiling, or vulnerability assessments. Unfortunately, integrating this mechanism into our simulation was not possible. Time limitations and insufficient data regarding user profiles and network interactions further complicate our ability to evaluate the effects of MTD techniques on quality of service (QoS). However, by incorporating this functionality into the simulator, we can further investigate the equilibrium between system security and quality of service while implementing MTD techniques.

Instead of routing requests through every possible service path in the network, an effective method is to route requests only through

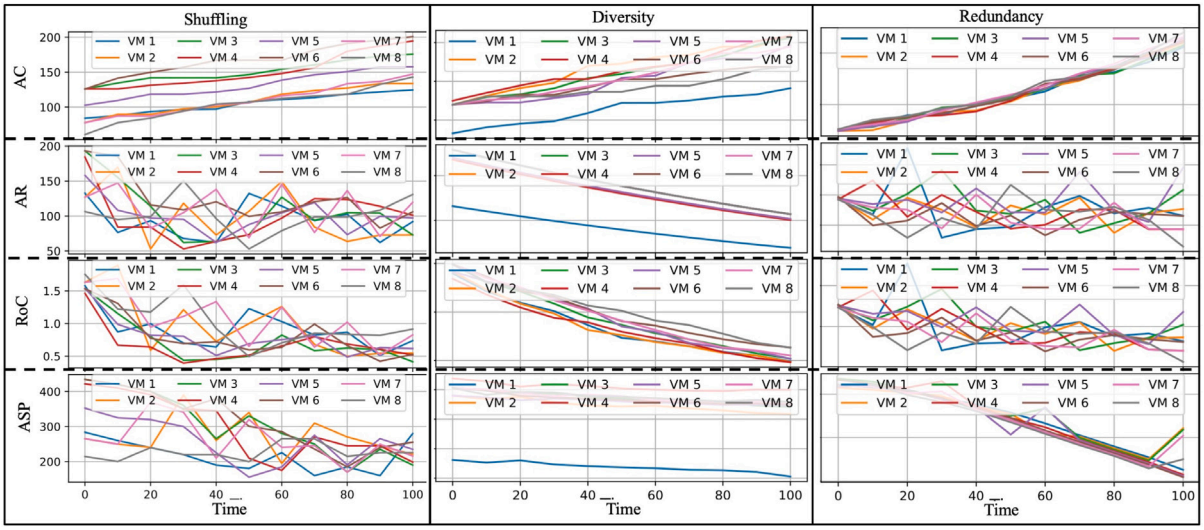


Fig. 9. Performance comparison of MTD shuffling, diversity, and redundancy strategies for VMs over time.

the shortest service paths, given the abundance of service paths (sp) within the network (defined as the path from the entry point VM in the network to the resource node). This may result in a decrease in bandwidth as well as latency. Let SSP be defined as the collection of the network's shortest service pathways (based on the higher layer of HARM modeling). As such, access latency can be measured by considering only the shortest paths.

6. Conclusion

This research greatly increased our understanding of MTD tactics for network security. Using the established framework, MTD approaches can be evaluated more effectively in the temporal domain, providing valuable insights that aid in developing strong defensive solutions. Network shuffle-based tactics frequently beat diversity-based ones in the study's rigorous comparison of several MTD methodologies. Notably, Shuffle and Diversity turns out to be the most successful tactic. The research highlights the need for more investigation of MTD techniques across various network configurations, assessment against sophisticated attacks, and a selective deployment approach to minimize resource consumption and service interruptions in real-world scenarios, even though it offers helpful guidance for building resilient defences. Given the fixed vulnerability inherent in OS and services, we advocate for the utilization of a dynamic model to effectively identify vulnerabilities within the IoT network. We concentrated solely on assessing Shuffle, Diversity, Redundancy, and their various combinations within MTD approaches. Exploring more MTD approaches and their combinations might increase the complexity of security analysis. Nevertheless, it might give cloud providers enhanced security solutions and defence capabilities. Cloud providers can strategically choose and implement preferred MTD approaches by evaluating the resources.

CRediT authorship contribution statement

Mohammed Tanvir Masud: Writing – original draft, Visualization, Validation, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Marwa Keshk:** Visualization, Validation, Supervision. **Nour Moustafa:** Supervision, Methodology, Writing, review & editing. **Benjamin Turnbull:** Supervision. **Willy Susilo:** Writing – review & editing.

Declaration of competing interest

There is no conflict of interst.

Acknowledgments

This research was supported under UNSW Canberra and Australian Research Council's Discovery Early Career Researcher Award (DECRA) (project number, DE230100116).

Data availability

No data was used for the research described in the article.

References

Alavizadeh, H., Aref, S., Kim, D.S., Jang-Jaccard, J., 2022. Evaluating the security and economic effects of moving target defense techniques on the cloud. IEEE Trans. Emerg. Top. Comput. 10 (4), 1772–1788.

Alavizadeh, H., Hong, J.B., Jang-Jaccard, J., Kim, D.S., 2018. Comprehensive security assessment of combined MTD techniques for the cloud. In: Proceedings of the 5th ACM Workshop on Moving Target Defense. pp. 11–20.

Alavizadeh, H., Hong, J.B., Kim, D.S., Jang-Jaccard, J., 2021. Evaluating the effectiveness of shuffle and redundancy mtd techniques in the cloud. Comput. Secur. 102, 102091.

Alavizadeh, H., Kim, D.S., Jang-Jaccard, J., 2020. Model-based evaluation of combinations of shuffle and diversity MTD techniques on the cloud. Future Gener. Comput. Syst. 111, 507–522.

Anwar, A.H., Kamhoua, C.A., 2022. Cyber deception using honeypot allocation and diversity: A game theoretic approach. In: 2022 IEEE 19th Annual Consumer Communications & Networking Conference. CCNC, IEEE, pp. 543–549.

Avizienis, A., 1985. The N-version approach to fault-tolerant software. IEEE Trans. Softw. Eng. (12), 1491–1501.

Aydeger, A., Saputro, N., Akkaya, K., 2019. A moving target defense and network forensics framework for ISP networks using SDN and NFV. Future Gener. Comput. Syst. 94, 496–509.

Baran, M.E., Wu, F.F., 1989. Network reconfiguration in distribution systems for loss reduction and load balancing. IEEE Trans. Power Deliv. 4 (2), 1401–1407.

Brown, A., Lee, T.-W., Hong, J.B., 2023. Evaluating moving target defenses against realistic attack scenarios. In: 2023 IEEE/ACM 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS). IEEE, pp. 1–8.

Cai, G.-l., Wang, B.-s., Hu, W., Wang, T.-z., 2016. Moving target defense: state of the art and characteristics. Front. Inf. Technol. Electron. Eng. 17 (11), 1122–1153.

Carter, K.M., Riordan, J.F., Okhravi, H., 2014. A game theoretic approach to strategy determination for dynamic platform defenses. In: Proceedings of the First ACM Workshop on Moving Target Defense. pp. 21–30.

Chen, H., Cam, H., Xu, S., 2021. Quantifying cybersecurity effectiveness of dynamic network diversity. IEEE Trans. Dependable Secur. Comput. 19 (6), 3804–3821.

Chlela, M., Mascarella, D., Joós, G., Kassouf, M., 2017. Fallback control for isochronous energy storage systems in autonomous microgrids under denial-of-service cyber-attacks. IEEE Trans. Smart Grid 9 (5), 4702–4711.

Cho, J.-H., Sharma, D.P., Alavizadeh, H., Yoon, S., Ben-Asher, N., Moore, T.J., Kim, D.S., Lim, H., Nelson, F.F., 2020. Toward proactive, adaptive defense: A survey on moving target defense. IEEE Commun. Surv. Tutor. 22 (1), 709–745.

Chong, F., Lee, R., Acquisti, A., Horne, W., Palmer, C., Ghosh, A., Pendarakis, D., Sanders, W., Fleischman, E., Teufel, III, H., et al., 2009. National cyber leap year summit 2009: Co-chairs' report. NITRD Program.

Common Vulnerabilities and Exposures (CVE), 2024. CVE Database. URL https://www.cve.org/. (Accessed 05 November 2024).

Compton, K., Hauck, S., 2002. Reconfigurable computing: a survey of systems and software. ACM Comput. Surv. (CsuR) 34 (2), 171–210.

Connell, W., Menascé, D.A., Albanese, M., 2017. Performance modeling of moving target defenses. In: Proceedings of the 2017 Workshop on Moving Target Defense. pp. 53–63.

Connell, W., Menasce, D.A., Albanese, M., 2018. Performance modeling of moving target defenses with reconfiguration limits. IEEE Trans. Dependable Secur. Comput. 18 (1), 205–219.

Danev, B., Masti, R.J., Karame, G.O., Capkun, S., 2011. Enabling secure VM-vTPM migration in private clouds. In: Proceedings of the 27th Annual Computer Security Applications Conference. pp. 187–196.

Enoch, S.Y., Mendonça, J., Hong, J.B., Ge, M., Kim, D.S., 2022. An integrated security hardening optimization for dynamic networks using security and availability modeling with multi-objective algorithm. Comput. Netw. 208, 108864.

Ergenc, D., Rak, J., Fischer, M., 2020. Service-based resilience for embedded iot networks. In: 2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, pp. 540–551.

Ge, M., Hong, J.B., Guttmann, W., Kim, D.S., 2017a. A framework for automating security analysis of the internet of things. J. Netw. Comput. Appl. 83, 12–27.

Ge, M., Hong, J.B., Guttmann, W., Kim, D.S., 2017b. A framework for automating security analysis of the internet of things. J. Netw. Comput. Appl. 83, 12–27.

Ge, M., Hong, J.B., Yusuf, S.E., Kim, D.S., 2018. Proactive defense mechanisms for the software-defined internet of things with non-patchable vulnerabilities. Future Gener. Comput. Syst. 78, 568–582.

Hobson, T., Okhravi, H., Bigelow, D., Rudd, R., Streilein, W., 2014. On the challenges of effective movement. In: Proceedings of the First ACM Workshop on Moving Target Defense. pp. 41–50.

Hong, J., Kim, D.-S., 2012. Harms: Hierarchical attack representation models for network security analysis.

Hong, J.B., Kim, D.S., 2013. Performance analysis of scalable attack representation models. In: Security and Privacy Protection in Information Processing Systems: 28th IFIP TC 11 International Conference, SEC 2013, Auckland, New Zealand, July 8-10, 2013. Proceedings 28. Springer, pp. 330–343.

Hong, J.B., Kim, D.S., 2015. Assessing the effectiveness of moving target defenses using security models. IEEE Trans. Dependable Secur. Comput. 13 (2), 163–177.

Hu, H., Wu, J., Wang, Z., Cheng, G., 2018. Mimic defense: a designed-in cybersecurity defense framework. IET Inf. Secur. 12 (3), 226–237.

Huang, Y., Ghosh, A.K., Bracewell, T., Mastropietro, B., 2010. A security evaluation of a novel resilient web serving architecture: Lessons learned through industry/academia collaboration. In: 2010 International Conference on Dependable Systems and Networks Workshops (DSN-W). IEEE, pp. 188–193.

Huang, Y., Huang, L., Zhu, Q., 2022. Reinforcement learning for feedback-enabled cyber resilience. Annu. Rev. Control. 53, 273–295.

Hutchins, E.M., Cloppert, M.J., Amin, R.M., et al., 2011. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. Lead. Issues Inf. Warf. Secur. Res. 1 (1), 80.

Jafarian, J.H., Al-Shaer, E., Duan, Q., 2015. An effective address mutation approach for disrupting reconnaissance attacks. IEEE Trans. Inf. Forensics Secur. 10 (12), 2562–2577.

Kim, S., Na, W., 2016. Safe data transmission architecture based on cloud for internet of things. Wirel. Pers. Commun. 86, 287–300.

Knight, J., Davidson, J., Nguyen-Tuong, A., Hiser, J., et al., 2016. Diversity in cybersecurity. Comput. 49 (04), 94–98.

Kolias, C., Kambourakis, G., Stavrou, A., Voas, J., 2017. DDoS in the IoT: Mirai and other botnets. Comput. 50 (7), 80–84.

Kolosok, I., Korkina, E., 2018. Decomposition of power system state estimation problem as a method to tackle cyber attacks. In: 2018 IEEE Industrial Cyber-Physical Systems. ICPS, IEEE, pp. 398–403.

Liaskos, C., Kotronis, V., Dimitropoulos, X., 2016. A novel framework for modeling and mitigating distributed link flooding attacks. In: IEEE INFOCOM 2016 - the 35th Annual IEEE International Conference on Computer Communications. pp. 1–9. http://dx.doi.org/10.1109/INFOCOM.2016.7524507.

Lv, Z., Chen, D., Cao, B., Song, H., Lv, H., 2023. Secure deep learning in defense in deep-learning-as-a-service computing systems in digital twins. IEEE Trans. Comput..

Maughan, D., 2009. National cyber security research assessment and roadmap. In: Proceedings of the 5th Annual Workshop on Cyber Security and Information Intelligence Research: Cyber Security and Information Intelligence Challenges and Strategies. 1–1.

National Vulnerability Database (NVD), 2024. NVD Vulnerability Database. URL https://nvd.nist.gov/vuln. (Accessed 05 November 2024).

Neshenko, N., Bou-Harb, E., Crichigno, J., Kaddoum, G., Ghani, N., 2019. Demystifying IoT security: An exhaustive survey on IoT vulnerabilities and a first empirical look on internet-scale IoT exploitations. IEEE Commun. Surv. Tutor. 21 (3), 2702–2733.

Nhlabatsi, A., Hong, J.B., Kim, D.S., Fernandez, R., Hussein, A., Fetais, N., Khan, K.M., 2018. Threat-specific security risk evaluation in the cloud. IEEE Trans. Cloud Comput. 9 (2), 793–806.

Penner, T., Guirguis, M., 2017. Combating the bandits in the cloud: A moving target defense approach. In: 2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing. CCGRID, IEEE, pp. 411–420.

Ramos, A., Lazar, M., Holanda Filho, R., Rodrigues, J.J., 2017. Model-based quantitative network security metrics: A survey. IEEE Commun. Surv. Tutor. 19 (4), 2704–2734.

Rehman, Z., Gondal, I., Ge, M., Dong, H., Gregory, M., Tari, Z., 2024. Proactive defense mechanism: Enhancing IoT security through diversity-based moving target defense and cyber deception. Comput. Secur. 139, 103685.

Schneier, B., 2015. Secrets and Lies: Digital Security in a Networked World. John Wiley & Sons.

Sengupta, S., Chowdhary, A., Sabur, A., Alshamrani, A., Huang, D., Kambhampati, S., 2020. A survey of moving target defenses for network security. IEEE Commun. Surv. Tutor. 22 (3), 1909–1941. http://dx.doi.org/10.1109/COMST.2020.2982955.

Sheyner, O., Haines, J., Jha, S., Lippmann, R., Wing, J.M., 2002. Automated generation and analysis of attack graphs. In: Proceedings 2002 IEEE Symposium on Security and Privacy. IEEE, pp. 273–284.

Sicari, S., Rizzardi, A., Grieco, L.A., Coen-Porisini, A., 2015. Security, privacy and trust in internet of things: The road ahead. Comput. Netw. 76, 146–164.

Sou, K.C., Sandberg, H., Johansson, K.H., 2012. Detection and identification of data attacks in power system. In: 2012 American Control Conference. ACC, IEEE, pp. 3651–3656.

Taguinod, M., Doupé, A., Zhao, Z., Ahn, G.-J., 2015. Toward a moving target defense for web applications. In: 2015 IEEE International Conference on Information Reuse and Integration. IEEE, pp. 510–517.

Tan, J., Jin, H., Zhang, H., Zhang, Y., Chang, D., Liu, X., Zhang, H., 2023. A survey: When moving target defense meets game theory. Comput. Sci. Rev. 48, 100544.

Xu, W., Higgins, M., Wang, J., Jaimoukha, I.M., Teng, F., 2023. Blending data and physics against false data injection attack: An event-triggered moving target defence approach. IEEE Trans. Smart Grid 14 (4), 3176–3188. http://dx.doi.org/10.1109/TSG.2022.3231728.

Yu, H., He, J., Zhang, T., Xiao, P., Zhang, Y., 2013. Enabling end-to-end secure communication between wireless sensor networks and the internet. World Wide Web 16, 515–540.

Zheng, Y., Li, Z., Xu, X., Zhao, Q., 2022. Dynamic defenses in cyber security: Techniques, methods and challenges. Digit. Commun. Netw. 8 (4), 422–435.

Zhuang, R., DeLoach, S.A., Ou, X., 2014a. Towards a theory of moving target defense. In: Proceedings of the First ACM Workshop on Moving Target Defense. pp. 31–40.

Zhuang, R., DeLoach, S.A., Ou, X., 2014b. Towards a theory of moving target defense. In: Proceedings of the First ACM Workshop on Moving Target Defense. pp. 31–40.

Zografopoulos, I., Ospina, J., Liu, X., Konstantinou, C., 2021. Cyber-physical energy systems security: Threat modeling, risk assessment, resources, metrics, and case studies. IEEE Access 9, 29775–29818.